AI INFRA · 面试复习

大模型推理框架与性能优化

18 题详解 + AI Infra 高频 150 问

原理 · 实现 · 性能诊断

术语约定：GDN 指 Gated DeltaNet，SSM state 指其矩阵递归状态；HBM 指 GPU 高带宽显存，GPU 算子的 IO 指显存与片上存储之间的数据搬运。文中“设计建议”“推导示例”为解释性内容，不代表框架已实现；保留原题号，原清单无第 19 题。

目录

1. nano-vLLM 和正式版 vLLM 有什么区别? 3

2. SGLang 里的 Replay SSM 机制了解吗? 3

3. Qwen3.5 的多模态输入在框架层需要做哪些扩展? 4

4. 多模态中的 ViT 更偏计算瓶颈还是 IO 瓶颈？为什么? 5

5. GDN 的 SSM state / conv state 在 Prefix Cache 场景下怎么处理？如果要支持应该怎么设计? 6

6. PD 分离主要解决什么问题？为什么要把 Prefill 和 Decode 分开? 7

7. Qwen3.5 Hybrid 做 PD 分离时，KV Cache 和 GDN state 应该怎么传输? 8

8. Decode 阶段的 IO 瓶颈主要出现在 Attention 还是 FFN? 9

9. MoE 和 Dense 模型有什么区别？MoE 的优势是什么? 10

10. TP、DP、EP 分别怎么切权重？TP 和 EP 同时使用时怎么拆分? 10

11. GEMM 为什么只能达到 cuBLAS 大约 76% 的性能？怎么用 Nsight Compute 定位? 11

12. CUDA Graph 解决什么问题？什么场景收益大，什么场景不明显? 13

13. Prefill 阶段是否也可能需要 CUDA Graph？判断依据是什么? 13

14. torch.compile 大概做了什么? 14

15. 算子融合为什么能提升性能? 14

16. 算子融合对 host gap 和显存/DDR 访存有什么改善? 15

17. vLLM 里还有哪些常见性能优化技术? 15

18. K3 的适配开始了吗？当前性能大概如何? 16

20. Continuous Batching 是怎么实现的？调度器每轮怎么组织 Decode 和 Prefill 请求? 18

补充面试问答：Kimi/K3 并行、CP、MTP、性能分析与工程基础 20

AI Infra 高频 100 问：腾讯混元优先 30

AI Infra 高频进阶 50 问（101–150）：混元部署与系统实战 43

复习时最值得记住的 9 句话 50

资料与源码索引 51

术语约定：GDN 指 Gated DeltaNet；下文将其矩阵递归状态称为 SSM state，以对应推理框架的命名。HBM 指 GPU 高带宽显存；讨论 GPU 算子的 IO 时，主要指显存与片上存储之间的数据搬运，不是磁盘 IO。

1. nano-vLLM 和正式版 vLLM 有什么区别?

直接回答：nano-vLLM 是适合学习和实验的精简推理引擎；vLLM 是面向广泛模型、硬件和服务负载的完整推理系统。区别在工程范围、调度复杂度和功能覆盖，不能简单概括为“一个快、一个慢”。 [S1–S3]

维度	nano-vLLM	vLLM
定位	用较少代码呈现推理核心路径	通用、高吞吐推理与在线服务
核心实现	已包含 KV 管理、Prefix Cache、TP、CUDA Graph 等关键机制	在这些基础上扩展更多后端、缓存策略、调度和分布式能力
模型覆盖	围绕较少模型和实现路径	多模型、多模态、MoE、Hybrid 等多种架构
调度	代码直观，策略和异常路径较少	Token 预算、抢占、分块 Prefill、推测解码、编码器预算等统一协调
工程复杂度	便于理解和改造	兼容性、并发、错误恢复、可观测性和部署能力更完整

一个值得讲出的源码差异：本次查看的 nano-vLLM `Scheduler.schedule()` 先从 waiting 队列安排 Prefill；只要这一轮排到了 Prefill，就返回 Prefill batch，否则才处理 Decode。它已经存在受限制的 Chunked Prefill：预算不足时只允许第一个请求分块。不要沿用旧印象，说它“一定没有分块 Prefill”。vLLM V1 则以每个请求尚需计算的 token 数为中心安排工作，第 20 题展开。[S2、S3]

面试收尾：我会用 nano-vLLM 理解最小闭环，再通过 vLLM 学习复杂模型、调度和生产部署。对比性能必须控制 GPU、模型、精度、输入输出长度、并发、缓存命中和优化开关，不能只看一个 tokens/s 数字。

2. SGLang 里的 Replay SSM 机制了解吗?

直接回答：ReplaySSM 将“每步更新并写回完整递归状态”改为“保存一个检查点状态和近期的紧凑更新记录”，多数 Decode 步直接算输出，周期性才把记录合并到状态。它减少大矩阵状态的写回，也让推测解码的回滚更便宜。它不是 CUDA Graph replay。 [S4、S5]

为什么值得做

以一个 head 的状态 `S ∈ R^(d_v×d_k)` 为例，标准 recurrent Decode 每步通常要读旧状态、计算更新、写新状态。状态元素数为 `d_v×d_k`，而一次低秩更新只需若干长度为 `d_v` 或 `d_k` 的向量和门控参数。状态较大时，反复读写矩阵的显存流量很可观。[S4]

核心流程

1. 保存某个历史位置的 checkpoint state `S_c`。
2. 每步把必要的更新信息写入短 ring buffer。Mamba 与 GDN 的更新式不同；GDN 需要与 delta rule、衰减相适配的记录，不能统一说成“只缓存原始 token”。
3. 在 checkpoint 与近期记录上计算当前输出；可以使用 output-only 路径，避免每步把新的完整状态落到显存。
4. 当窗口到达阈值，或需要导出完整状态时，执行 flush/materialize：合并近期更新，得到新的 checkpoint，再重置窗口。[S4、S5]

一个便于理解的线性代数例子：如果某次更新为 $v\ k^\top$，它对输出的贡献可以算成 $v(k^\top q)$，不用先生成整个外积矩阵。GDN 带有状态相关的 delta correction，实际算法比这个示例复杂；这个等式解释的是“只求输出可能比构造整个状态更便宜”。[S4]

收益和边界

- **降低状态写流量**。大多数步仍可能读取 checkpoint；不能说成“状态读写全部消失”。实际收益还取决于 buffer 读取、重放计算和 flush 成本。
- **便于 speculative rollback**。拒绝的 draft 不必对应一份完整矩阵快照；可以截断未提交记录，再提交 accepted prefix。具体实现可能每次 commit 都 fold，也可能允许已接受记录继续留在 buffer，必须看版本。
- **有机会并行验证多个 token**。通过 checkpoint 加记录计算输出，减少逐 token 物化状态造成的串行依赖。
- **浮点结果不保证逐位一致**。代数等价不代表计算顺序、精度变化后 bitwise 相同，需要误差和模型质量验证。[S4、S5]

SGLang 的接入涉及 GDN backend、状态池、Decode/verify kernel 和 accepted-prefix commit。本次检索到的开发记录使用 `--enable-linear-replayssm`、`--enable-linear-replayssm-spec` 等参数；它们和 MTP、树状 draft、Radix Cache、状态回收之间的组合支持需要按实际版本检查，不能把 RFC 或单个 PR 当作所有发布版的保证。[S5]

与后面两题的关系：逻辑状态不一定等于 checkpoint tensor。当存在尚未 fold 的已接受更新时，完整逻辑状态是“checkpoint + 有效 replay 记录”。做 Prefix Cache、PD 传输或 CPU 备份时，应该先 materialize，或者连同记录、位置和有效长度一并保存。这是从其表示方式直接推出的正确性要求。

3. Qwen3.5 的多模态输入在框架层需要做哪些扩展？

直接回答：要把纯文本的 `token_ids → LLM` 扩展成“多模态解析与预处理 → 视觉编码 → embedding 合并与位置编码 → Hybrid LLM”，并让调度器和缓存系统理解视觉计算及 GDN 状态。仅增加一个 ViT 类不够。[S6–S8]

Qwen3.5 包含不同规格；以下讨论模型卡和 vLLM 中的图像、视频输入路径，不把音频能力自动推定给所有 Qwen3.5 模型。[S6、S7]

框架层	需要扩展的内容	容易遗漏的问题
请求协议	接收图像、视频及其顺序；接入对应 chat template 和 processor	占位符顺序与媒体列表顺序必须一致
CPU 预处理	解码图像/视频，采样帧，resize、normalize、patchify	动态分辨率、帧数会改变视觉 token 数和成本
输入表示	保留 `pixel_values`、`image_grid_thw`、视频对应字段及必要时间信息	不能只传一个扁平 token 列表
模型执行	注册 vision tower、视觉特征合并/投影和语言模型入口	按实际 config 处理视觉结构，不能机械复制另一个模型的 DeepStack
Embedding 合并	将视觉特征放入正确的多模态 token 位置	特征数量必须与占位 token 范围匹配
位置编码	使用模型的多模态 RoPE、图像/视频多维位置规则，保持后续 Decode 连续	分块 Prefill 和缓存恢复后的位置必须一致
调度	增加 encoder 计算预算、encoder cache 与执行依赖	Decode token budget 不能代表视觉编码成本
缓存与内存	同时管理视觉特征、Attention KV、GDN state/conv state	Prefix key 不能只包含相同的占位符 token
并行与图捕获	支持视觉编码器的并行模式、动态 shape 分桶及必要的 eager 路径	图像大小变化可能触发重新编译或图捕获不命中

上述表格是结合官方多模态接入接口与 Qwen3.5 源码归纳的扩展清单。vLLM 当前模型实现复用了部分 Qwen3-VL 视觉/处理接口，同时语言部分接入 Qwen3.5 的 Hybrid 层和 GDN 状态接口；复用现有组件通常比另写一整套输入栈更合理。[S7、S8]

缓存正确性示例：两个请求都有 `<image>` 占位符，但图片不同，不能复用图片位置之后的同一段语言状态。建议缓存标识纳入媒体内容标识、预处理参数、模型/适配器版本和影响位置编码的元数据；视觉特征缓存与 LLM Prefix Cache 分别管理。

4. 多模态中的 ViT 更偏计算瓶颈还是 IO 瓶颈？为什么？

直接回答：只看 GPU 上的 ViT 主干，在视觉 token 较多、batch 合理时，通常更偏计算瓶颈；小输入、小 batch 或未优化的实现可能受显存访存、kernel launch 限制。把图片读取、解码和传输算进去后，整个视觉流水线也可能受 CPU 或 IO 限制。[S9、S10]

ViT 的 QKV 投影、Attention 和 MLP 处理一批 patch tokens，矩阵乘的 `M` 往往比单 token Decode 大。权重能被多个 token 复用，算术强度更高，更容易发挥 Tensor Core 能力。MLP 和投影大致随 token 数线性增长；全局 Attention 的部分计算随 token 数平方增长，具体还取决于窗口化等结构。[S9]

判断不能只看模块名称，要看测量：

- Tensor Core/SM 吞吐较高、DRAM 带宽没有接近上限：偏计算受限。
- DRAM 吞吐高、算术强度低、访存等待明显：偏显存带宽受限。

- GPU 时间线有大量空白、CPU 预处理忙或 H2D 拷贝没有重叠：偏端到端输入流水线受限。
- 小矩阵、大量短 kernel、GPU 吃不满：可能是 launch 或并行度不足，并非纯粹的“计算量太大”。[S10、S11]

面试表述：“ViT 在大 patch batch 下通常比 Decode 更偏 compute-bound，但我会把 CPU 预处理、H2D、GPU encoder 分开测；‘多模态是 IO 瓶颈’不是对 ViT 本身的充分判断。”

5. GDN 的 SSM state / conv state 在 Prefix Cache 场景下怎么处理？如果要支持应该怎么设计？

直接回答：Attention KV 是沿序列位置追加的数据，GDN state 是前缀历史压缩成的递归状态。要从位置 `p` 继续执行，必须同时拿到覆盖该前缀的 KV、恰好在 `p` 的 SSM state，以及对应的 conv state。只有最终状态，不能自然恢复任意较早的前缀。 [S7、S12、S13]

两种 state 分别是什么

- **SSM state：**每个 GDN 层、每个相关 head 的矩阵状态，概念尺寸通常为 `[H_v, d_k, d_v]`，物理维度顺序和 TP 分片由实现决定。
- **conv state：**短因果卷积继续计算所需的历史输入。逻辑上需要保留最近 `kernel_width-1` 个位置；实际缓存可能保留 `kernel_width` 个槽位或额外 speculative 空间，不能硬编码成一种物理形状。[S7、S13]

“SSM state 固定大小”只表示**单个时间点**的状态大小不随上下文长度增长。若对每个 Prefix block 都存一份完整状态，总内存仍会随检查点数量增长。

建议的缓存结构

```
PrefixCheckpoint {
    prefix_identity           # token + 多模态身份 + 模型/适配器等
    end_position              # 已处理完的 token 数，统一定义边界
    attention_block_refs[]    # 全部 Attention 层到该边界的 KV
    gdn_state_refs[]          # 全部 GDN 层在该边界的 recurrent state
    conv_state_refs[]         # 同一边界的卷积历史
    layout_version, dtype, tp_mapping
    refcount, ready_event
}
```

这是设计建议，关键不变量是：**所有层对应同一个已提交的 token 边界**。如果存在 ReplaySSM，还需让该检查点已 materialize，或者显式保存完整 replay 描述。

插入、命中和淘汰流程

1. **选择检查点粒度。**在块边界、Prefill chunk 边界或请求可复用边界保存状态。不能假设 Prefill kernel 自动输出了每个 token 的中间状态；需要 kernel 暴露选定边界，或把计算切到这些边界。
2. **原子发布。**KV 与所有 GDN/conv 状态写完后才将节点标成可命中，异步 copy 通过事件保证先后顺序。

3. **按共同边界匹配**。如果 Attention KV 命中 1024 token，而完整 GDN 检查点只到 768，最直接的实现从 768 恢复，计算剩余部分。不能只因 KV 命中 1024 就跳过所有层的这 256 token。
4. **保护共享状态**。缓存快照不可被后续 Decode 原地修改。命中后复制到请求的可写槽，或实施 copy-on-write；KV 完整块可以引用计数共享。
5. **协调生命周期**。跟踪不同资源的引用和可达性，防止出现“KV 还在但状态已淘汰、节点却仍标记可完整命中”的悬空命中。
6. **控制内存**。可稀疏保存状态、在热点前缀多保存、对低收益检查点淘汰；稀疏保存意味着多算一段后缀。[S12 提供 Hybrid 缓存管理背景；以上为工程设计建议。]

可以有更复杂的方案吗？

可以做分层命中：某些 Attention 层复用更长的 KV，同时为 GDN 重算一段。但跨层 hidden states 存在依赖，不能仅修改一个命中长度字段就实现。另一方案是保存中间激活或可重放更新记录，以空间、重放时间换取更细粒度恢复。

正确性测试重点：整段计算与命中后续算的 logits/state 对比；共享前缀后分叉；刚好命中块边界与跨边界；不同图片、LoRA、位置参数；淘汰后恢复；推测解码拒绝回滚；ReplaySSM buffer 尚未 flush 的情况。浮点路径不同用合理容差，并补充输出质量验证。

6. PD 分离主要解决什么问题？为什么要把 Prefill 和 Decode 分开？

直接回答：PD 分离主要解决 Prefill 与 Decode 混跑时的资源争抢和延迟干扰，并允许两阶段独立配置资源、独立扩容。目标常是同时改善 TTFT 与逐 token 延迟的服务质量，而不只是追求单机峰值吞吐。 [S14]

阶段	典型工作	典型资源特征	主要体验指标
Prefill	处理输入 token，建立 KV 和递归状态	大矩阵乘，通常计算密集	TTFT：首 token 延迟
Decode	根据已有上下文逐步生成	小 batch 时权重读取、长上下文 KV 读取显著	ITL/TPOT：逐 token 延迟

在同一 GPU 上，长 Prefill 会占用执行资源和时间，正在 Decode 的请求可能等很久才得到下一 token。分离后，P 池承接输入计算，D 池承接稳定生成，并可根据输入/输出长度分布、并发和 SLO 调整比例。[S14]

代价也必须说明：增加状态传输、网络和控制面开销，通常需要两端各有模型权重，也会出现排队不均衡。可按以下关系评估：

> 收益取决于：减少的混跑干扰 + 更合适的资源配置
> 是否超过状态交接成本 + 额外排队与资源闲置

短 prompt、低负载、互联较慢时，分离不一定更划算。Chunked Prefill 和混合 batching 也能缓解干扰；PD 分离是更强的部署层手段。[S14、S15]

7. Qwen3.5 Hybrid 做 PD 分离时，KV Cache 和 GDN state 应该怎么传输?

直接回答：传输一个完整的“可继续解码的请求状态”：Attention 层传到交接边界的 KV，GDN 层传同一边界的 SSM state 和 conv state，并带上位置、布局、分片及生命周期元数据。不能只传 KV。 [S7、S12、S16]

传什么

数据	随上下文长度的增长	交接要求
Attention K/V	通常线性增长	覆盖每个 Attention 层所需前缀；明确尾块有效长度
GDN SSM state	单个边界固定大小	全部 GDN 层、相同已提交边界
GDN conv state	单个边界固定大小	与 recurrent state 的位置一致
ReplaySSM 记录（如启用）	随有效窗口增长	先 materialize，或传 checkpoint、记录、游标和有效范围
控制元数据	较小	request ID、token 边界、位置编码信息、dtype/layout、rank 映射等

视觉输入已经完整 Prefill 完毕后，D 通常依靠这些语言模型状态继续生成，无需重复传原图和重跑 ViT。如果交接发生在未完成的多模态输入中，或者协议支持后续追加媒体，还要单独设计视觉特征和输入依赖的交接。[S7、S8；后一句为设计推论。]

建议的交接协议

1. **D 先分配目标资源并返回 descriptor**。包括本地 KV 页映射、状态槽和传输目标；不能直接把 P 的物理 block ID 当成 D 的 block ID。
2. **P 完成至边界 `T`，固定快照**。例如 `T` 表示已经消费 `x[0:T]`，所有状态必须遵守同一约定。
3. **异步搬运**。KV 可按块提前传，GDN 最终状态需等对应计算完成；使用部署支持的 GPU 通信/传输后端，借助事件管理计算流和传输流依赖。
4. **全部组件就绪后提交 handoff**。D 确认同一请求、同一版本、同一边界的数据完整，再进入 Decode 队列。
5. **D 确认接管后释放 P 的引用**。处理取消、超时、重试和重复消息，避免写后即释放造成传输读到复用内存。[S16 为传输后端背景；此流程为设计建议。]

首 token 的边界尤其容易错：若 P 已消费全部 prompt 并采样出第一个输出 token y_0，但尚未把 y_0 输入模型，则传出的状态仍只覆盖 prompt。D 下一次应消费 y_0。不能因为消息携带了 y_0，就宣称 KV/SSM 已包含它。

TP 相同与不同

- **相同 TP 与缓存布局：**优先让对应 rank 传对应 shard，通常最简单。

- **不同 TP：**需要明确 KV heads、GDN heads/通道、conv 通道在两端的逻辑切分，进行 gather/scatter 或 all-to-all 等重分片。GQA 可能存在 KV head 复制，不能只按 tensor 字节数平均切。
- **支持边界：**本次核对的 vLLM NIXL 兼容矩阵明确要求 Hybrid SSM/Mamba 使用 `P TP == D TP`，Mamba 层暂不支持异构 TP，并且 Hybrid Memory Allocator 不支持两端使用不同 block size。应优先采用支持的同 TP、同 block size 配置，把异构 TP 视为需要额外开发和验证的能力。[S16]

推导示例：GDN state 并不一定“小到可以忽略”

Qwen3.5-397B-A17B 模型卡给出 60 层，其中 45 层 GDN、15 层 Attention；GDN 有 64 个 value heads、head dimension 128；Attention 有 2 个 KV heads、head dimension 256。[S6]

假设 GDN state 为 FP32、KV 为 BF16，不考虑 TP 复制、对齐、conv state 和其他缓存：

```
GDN 状态总量 = 45 × 64 × 128 × 128 × 4 bytes = 180 MiB
Attention KV = 15 × 2(K/V) × L × 2(KV heads) × 256 × 2 bytes
             = 30 KiB × L
L = 8192 时，KV = 240 MiB
```

这是由公开形状和假设精度推导的逻辑数据量，不是实测，也不是所有 Qwen3.5 型号的数值。可见短上下文下递归状态可能占交接数据的大头；长上下文下 KV 才逐渐主导。

8. Decode 阶段的 IO 瓶颈主要出现在 Attention 还是 FFN?

直接回答：不能固定归因。小 batch、短上下文时，FFN/投影的权重读取经常很重要；长上下文或更大 batch 下，Attention 读取 KV Cache 的成本可能占主导。Hybrid 模型还要考虑 GDN state 的读写。 [S9、S12、S17]

从每轮数据量理解：

```
FFN 权重流量 ≈ 本轮需要读取的 FFN 权重字节数
Attention KV 流量 ≈ 2 × Σ请求上下文长度 × KV_heads × head_dim × KV字节数
```

上式为单 Attention 层的一阶估算，忽略缓存命中、分页、数据重复读取等实际影响。Dense 的同一块权重可以在 batch 内被多个 token 复用；增大 batch 后，每个输出 token 分摊的权重流量降低。不同请求的 KV 通常不能像模型权重那样直接共享读取，长上下文会使其成本增加。[S9、S17]

FFN 参数通常较多，小 batch 下其 GEMM 接近矩阵向量乘，权重读一遍只做少量复用。Attention 还含 QKV/O 投影，本身也有权重流量；比较时要说明“Attention”指完整模块还是仅注意力核心 kernel。

MoE 还要额外看：本轮实际触及的专家数、每个专家收到多少 token、专家权重读取和 EP 通信。不应只用单 token 激活参数量解释大 batch 的总流量。

定位方法：先看每类 kernel 的总耗时占比，再看其 DRAM/L2 流量与吞吐；扫描 batch size 和 context length。如果耗时随上下文长度明显增长，Attention KV 是重点；如果增加 batch 后每 token 成本显著降低而短上下文仍慢，应检查权重搬运和小 GEMM 利用率。[S10、S11]

9. MoE 和 Dense 模型有什么区别？MoE 的优势是什么？

直接回答：Dense 对每个 token 执行同一套稠密网络；MoE 在部分层使用 router 为 token 选择少量专家，只激活总专家参数中的一部分。其主要优势是在较低的单 token 激活计算量下提供更大的总参数容量。 [S18、S19]

通常保留 Attention 等共享结构，把 FFN 替换为多个专家，router 选择 top-k 专家并按权重组合结果；某些架构另有始终启用的 shared expert。对于同一层，单 token 的专家计算量主要与选中的专家数量和大小相关，而不是全部专家数。[S18、S19]

比较必须说明基准：

- **同总参数量：**MoE 通常激活更少参数，单 token 计算更低。
- **同激活计算量：**MoE 可容纳更大的总参数容量，但质量收益需要实际评测。
- **部署成本：**全部专家权重仍需存放在 GPU、CPU 或其他存储层；参数容量没有凭空消失。

实际劣势包括 router 和 token 重排成本、专家负载不均、跨卡 dispatch/combine、每个专家 token 太少导致 GEMM 利用率低，以及更复杂的部署。**MoE 的“低激活 FLOPs”不自动等于“更低延迟”**；互联、负载均衡和 batching 决定理论收益能兑现多少。[S18、S19]

10. TP、DP、EP 分别怎么切权重？TP 和 EP 同时使用时怎么拆分？

直接回答：TP 切同一个层/矩阵；DP 在不同副本之间切请求；EP 按专家分配权重。TP 和 EP 联用时，可以先把专家分组放到不同 EP rank，再把同一个专家的矩阵沿 TP 维切分，但必须确认实际框架采用的进程组。 [S19、S20]

并行方式	权重如何分布	主要通信
TP，Tensor Parallel	一个逻辑层的权重分片到多卡；每卡参与同一批 token 的计算	All-reduce、reduce-scatter、all-gather 等
DP，Data Parallel	纯 DP 推理中，各副本持有完整逻辑模型；副本内部可再做 TP/EP	请求分发；纯独立推理副本通常无需每层同步
EP，Expert Parallel	不同 rank 持有不同专家，token 路由到对应 rank	Dispatch/combine，常表现为 all-to-all 或专用通信

训练中的 DP 还需要梯度同步；这里主要讨论推理。[S20]

TP 的矩阵例子

采用 `X[M,H] × W[H,F]` 的记号：

```
Column Parallel: 沿 W 的 F 维切
W = [W0 W1 ...]，每卡算一部分输出列。

Row Parallel: 沿 W 的 H 维切
各卡计算 Xi × Wi，再对部分结果求和。
```

Transformer FFN 通常把上投影按输出维切、下投影按输入维切。SwiGLU 的 gate/up 分片需要对应；Attention 常按 head 分片，GQA 的 KV heads 数少时还可能涉及复制。[S20]

TP=2、EP=4 的概念例子

假设明确建立一个 **2×4 的二维进程网格**、共有 16 个 routed experts：

- 4 个 EP 组分别负责 experts 0–3、4–7、8–11、12–15。
- 每个 EP 组内部用 2 张卡切其专家矩阵；同一专家的一次计算由这 2 个 TP rank 合作完成。
- token 先路由到拥有该专家的 EP 组，再在该组内完成 TP 计算，最后组合输出。

这个例子用了 8 张卡，但**不能把所有框架都理解成 `总卡数 = TP×DP×EP`**。

在 vLLM 的典型 MoE EP 部署中，启用 EP 后，专家并行组会结合 TP 与 DP 的 rank；文档用 `EP size = TP size × DP size` 描述相关配置。MoE 专家层采用 EP，不代表每个专家必然又被切成 TP 份；稠密部分与专家部分可采用不同分组。shared expert、router 的处理也要单独核对。[S19]

面试收尾：先问清楚“TP 是只用于 Attention/稠密层，还是也用于专家矩阵”，再画进程组和权重布局；仅报 TP、DP、EP 三个数字不足以唯一确定分布。

11. GEMM 为什么只能达到 cuBLAS 大约 76% 的性能？怎么用 Nsight Compute 定位？

直接回答：76% 只是基准结果，不能仅凭这个比例判断原因。需要先保证比较公平，再用时间线和 kernel 指标区分 Tensor Core 利用、数据搬运、并行度、访存延迟及尾块浪费。cuBLAS/cuBLASLt 会针对大量形状和硬件选择不同算法。[S9–S11、S21]

先确认“76%”是什么意思

如果吞吐 `P_ours / P_cuBLAS = 0.76`，相同工作量下：

```
T_ours / T_cuBLAS = 1 / 0.76 ≈ 1.316
```

即延迟约高 31.6%，不是只慢 24%；也不表示 GPU 利用率是 76%。

第一步：检查基准是否公平

必须一致：GPU、M/N/K、dtype、累加精度、TF32 开关、布局和转置、alpha/beta、epilogue、是否需要额外输出转换，以及工作区限制。先校验数值正确性，再充分 warm-up，使用 CUDA events 计 GPU 时间，分开统计初始化和编译，重复测量稳定区间。cuBLAS 若使用更多 SM 工作分解或 split-K 等算法，也要计入其完整执行序列。[S9、S21]

第二步：先用 Nsight Systems 看“慢在哪里”

```
nsys profile --trace=cuda,nvtx,osrt -o gemm_timeline ./gemm_bench
```

确认比较的是 kernel 本身，还是被 CPU 发射间隔、同步、内存分配、H2D/D2H 或额外 pack/transpose 拖慢。若绝大部分时间在 GPU kernel 外，直接调 tile 可能解决不了主要问题。[S11]

第三步：用 Nsight Compute 看目标 kernel

```
# 调整 kernel 过滤名称及 skip 数，使其对应你的 warm-up 和正式迭代。
ncu --set full --kernel-name 'regex:my_gemm' \
    --launch-skip 20 --launch-count 1 -o my_gemm_report ./gemm_bench

# 不同版本/架构指标名称可能不同，先查询可用项。
ncu --list-sections
ncu --query-metrics
```

NCU 会为采集指标执行 replay，采样运行的整体时长不适合作为线上吞吐结果；基准计时与 profiler 诊断要分开。[S10]

观察项	可能说明什么	下一步实验
Tensor/Compute 吞吐低，DRAM 也不高	并行度不足、依赖链长、小 grid、同步等待	调 CTA/warp 数、tile、流水级数；检查是否有足够 CTA
DRAM 吞吐高，算术强度低	重复加载、数据复用不足、输出/辅助流量大	改 tile 复用、访存合并、减少中间张量
Long scoreboard 等访存依赖等待明显	global/local load 延迟没被隐藏	检查合并访问、spill、预取和多级流水
寄存器/SMEM 占用限制驻留 CTA	tile 或流水过大，资源使用压低并行度	减 tile/流水级数，平衡复用与 occupancy
Local memory load/store 多	可能存在寄存器 spill	减少 live range、tile 或临时变量
Shared memory 冲突、wavefronts 偏多	SMEM layout 与 warp 访问不匹配	padding、swizzle、重新安排布局
Barrier/同步等待高	流水衔接或不同 warp 工作不均	调整生产消费分工和同步粒度

观察项	可能说明什么	下一步实验
大量 tail/不满 wave	M/N 边界浪费或 CTA 数与 SM 数不匹配	shape 专用 tile、split-K/stream-K 等分解

该表是基于 NCU 指标的诊断路径，不是“看到某个 stall 就一定对应某个根因”。应结合 roofline、资源用量、源码关联和消融实验验证。[S10、S21]

常见的 cuBLAS 优势

高度适配的 tile 与调度、Tensor Core 指令选择、架构相关的异步搬运与多级流水、共享内存布局、epilogue 融合，以及针对小矩阵/不规则形状的特殊实现。不能假设自己的一个通用 kernel 能覆盖其所有算法优势。[S21]

最终输出应是证据链：“目标 shape 下，A 指标和 B 时间占比表明瓶颈是 C；改变 D 后目标指标改善，数值正确，稳定延迟从 X 降到 Y。”不要写成“因为 occupancy 不够，所以慢 24%”。

12. CUDA Graph 解决什么问题？什么场景收益大，什么场景不明显？

直接回答：CUDA Graph 将重复的 GPU 工作及依赖捕获成图，后续用较低 CPU 开销提交，减少逐 kernel 发射和调度造成的间隔。它主要优化执行提交，不会自动减少模型 FLOPs，也不会自动融合 kernel。[S22、S23]

收益通常较大：每轮有很多短 kernel；Decode 步反复执行类似路径；CPU 发射开销占比高；可用稳定地址和少量 batch/shape 桶覆盖大部分请求。[S22、S23]

收益不明显或可能被抵消：一次执行本身是很长的大 GEMM；受 HBM 或互联带宽限制且 GPU 已连续忙碌；shape/控制流变化频繁导致大量 eager fallback；图捕获和额外内存成本较高；为了匹配图尺寸产生过多 padding。[S22、S23]

约束要说准确：普通静态捕获通常要求可复用的执行结构和稳定的相关内存地址，但输入数据内容可以在 replay 前更新。动态 workload 可以用分桶、padding、分段图和部分 eager 执行处理；不是“输入 token 必须完全相同”。[S23]

13. Prefill 阶段是否也可能需要 CUDA Graph？判断依据是什么？

直接回答：可以。是否值得用 Graph 取决于 launch/host 开销在 Prefill 总时间中的比例，以及 shape 能否稳定复用，不取决于阶段名字。短 prompt、小 Prefill chunk 和包含很多小算子的路径可能获益。[S22、S23]

长 Prefill 通常大 GEMM 较多，GPU 计算时间足以掩盖一部分 CPU 发射开销，Graph 的相对收益可能小。若大量短请求或固定 chunk 重复出现，图捕获可以减少每轮调度间隙。动态长度、视觉输入和某些 Attention backend 可能需要分段捕获或混合 eager 路径。[S23]

可操作的判断流程：

1. 用时间线统计 eager 下的 GPU 空隙和 host 发射成本。
2. 看 token 数、batch、视觉 shape 的分布，评估少量 buckets 的覆盖率。
3. 分别测试 eager、分段图、可支持的完整图；计入 padding、图内存和捕获成本。
4. 在真实负载下比较 TTFT、吞吐及尾延迟，而不是只测一个固定 shape。

推导示例：若可消除的发射间隙只占原时间 3%，即使全部消除，理论加速也仅 `1/0.97≈1.031`；若占 25%，上限约为 `1/0.75≈1.333`。这是 Amdahl 定律的估算，实际还要扣除 Graph 本身和 padding 开销。

14. torch.compile 大概做了什么？

直接回答：它把 Python/PyTorch 的一部分执行捕获为计算图，在满足输入和环境 guards 的条件下复用编译结果，并通过后端生成或选择更高效的执行代码。常见流程包含 TorchDynamo、图变换和 TorchInductor。 [S24]

```
Python / PyTorch 程序
    ↓ TorchDynamo：捕获可支持的执行，建立 FX 图与 guards
图分解、函数化、形状与其他图级处理
    ↓ 训练时还需要自动微分相关图处理
TorchInductor 等后端：调度、融合、代码生成/外部库调用
    ↓
GPU kernels（常见 Triton 或库算子）/ CPU 代码
```

AOTAutograd 在训练编译中处理前向、反向等图；不要把纯推理概括成“先编译 backward”。实际编译路径取决于后端和模式。[S24]

它能做的优化包括：减少部分 Python 调度成本、融合可融合算子、消除部分中间张量、优化布局和生成特定 shape 的代码。它也可能保留对高性能外部 GEMM 库的调用，而非自己生成每一个算子。[S24]

边界：不支持的 Python 行为可能导致 graph break；guards 不满足可能触发重新编译；首次编译有时间和内存成本；动态 shape 不等于不再需要 guards；自定义算子有时会限制跨算子优化。`torch.compile` 与 CUDA Graph 互补，某些配置会组合使用，二者并非同一个功能。[S23、S24]

15. 算子融合为什么能提升性能？

直接回答：融合可减少 kernel launch、中间张量的显存读写及重复工作，使前一个操作的结果直接在寄存器或共享内存中交给下一个操作。 [S24、S25]

例如 `y = gelu(x + bias)`。假设有 `N` 个元素、每元素 `b` 字节，忽略 bias 广播流量：

```
分开执行：读 x + 写 temp + 读 temp + 写 y ≈ 4Nb
融合执行：读 x + 写 y                    ≈ 2Nb
```

同时从两次 kernel launch 变成一次。这个估算表示全局内存访问的逻辑流量；实际 HBM 流量还受 L2 命中等影响，不能直接承诺带宽或总时间减半。

常见例子：bias+activation、residual+RMSNorm、SwiGLU 的 pointwise 部分、GEMM epilogue；FlashAttention 则通过分块和融合，避免显式物化完整 Attention score/probability 矩阵，是更强的 IO 感知算法优化。[S17、S24、S25]

融合不是越大越好。过多 live values 会增加寄存器使用甚至 spill；资源占用可能降低并行度；不合适的融合会妨碍使用更快的专用 GEMM。应比较融合后整体延迟，而不是只数 kernel 数量。[S10、S25]

16. 算子融合对 host gap 和显存/DDR 访存有什么改善?

直接回答：对 host gap，融合减少需要 CPU 提交的 kernel 数；对显存，融合减少跨 kernel 中间结果的读写与分配。两类收益要分别测量。 [S22、S24、S25]

观察维度	融合后的典型变化	不应该误解为
CPU/host	Python、dispatcher 和 CUDA launch 调用次数减少	一定消除所有 GPU 空隙
GPU 执行间隙	相邻小 kernel 之间的一些发射间隔被去掉	修复所有同步、网络或 CPU 预处理等待
寄存器/SMEM	中间结果可在片上直接复用	片上存储无限、没有资源代价
GPU 显存	减少中间 tensor 的 global store/load 与峰值容量	权重和最终输出都不再需要访存
CPU DDR	仅当改变 CPU 计算或数据搬运路径时才可能减少	GPU kernel 融合必然减少主机内存流量

在独立显卡上，应把主机 DDR 与 GPU HBM/GDDR 分开。GPU 算子之间的中间张量通常在 GPU 显存中，不是每次都绕回 CPU 内存。统一内存或 CPU offload 场景还需检查迁移与总线流量。[S10、S25]

与 CUDA Graph 的区别：Graph 减少多次提交的开销，通常仍执行多个 kernel；融合真正改变 kernel 边界，并可能消除中间存储。Graph 中的操作还受依赖关系约束，不能说成“合成图后自动并行”。[S22、S23]

17. vLLM 里还有哪些常见性能优化技术?

直接回答：可以按缓存、调度、算子、执行和分布式几个层面归纳，重点说明每项解决的瓶颈及代价。 [S12、S15、S17、S19、S23、S26]

技术	主要解决的问题	需要关注的代价/边界
PagedAttention / 分页 KV	连续大块 KV 分配造成的浪费，提高可容纳并发	页表与后端访问成本；不是消除 KV 数据本身
Automatic Prefix Caching	重复前缀的重复 Prefill 计算	命中率、内容身份、缓存容量；Hybrid 还需状态对齐
Continuous Batching	固定 batch 中短请求结束后的槽位空闲	调度开销、公平性、抢占和延迟目标
Chunked Prefill	长 Prefill 干扰 Decode、单轮预算失衡	chunk 太小会降低大 GEMM 效率

技术	主要解决的问题	需要关注的代价/边界
FlashAttention / 优化 Attention 后端	Attention 的 IO、并行度和数据布局	GPU/shape/dtype/backend 的适配范围
权重量化、KV 量化	权重/缓存容量与带宽压力	精度、反量化成本、kernel 支持及校准
CUDA Graph	CPU 发射与 kernel 间隙	捕获覆盖率、静态 buffer、padding 和图内存
torch.compile / 算子融合	Python 开销、pointwise 访存和执行碎片	编译成本、重编译与自定义算子边界
Speculative Decoding / MTP	自回归逐步串行导致的低效率	草稿成本、接受率、验证成本和额外状态
TP / PP / DP / EP	模型放置、单副本延迟或系统吞吐	通信、流水气泡与负载不均
MoE grouped/fused GEMM、负载均衡	专家碎片化计算与热点专家	重排/通信成本，负载随请求变化
通信计算重叠、异步调度	GPU 等 CPU 或等待通信	有效重叠依赖数据依赖和资源竞争
PD 分离与缓存传输	Prefill/Decode 的资源争抢	网络、交接成本和资源配比
多模态 encoder/processor cache	相同媒体重复处理或编码	缓存身份、容量、shape 与生命周期

这些技术并非每个模型、GPU、后端都能同时启用。面试时可以选择与瓶颈匹配的三四项展开；上线时先确定 TTFT/ITL/吞吐目标，再基于测量逐项验证。[S12、S15、S19、S23、S26]

18. K3 的适配开始了吗？当前性能大概如何？

直接回答：K3（Kimi 系列）的适配已经启动，目前处于模型结构打通、功能正确性验证和性能优化并行推进的阶段。按当前相同模型、精度、输入输出长度、并发和并行配置的阶段性对标结果，性能大约为 H20 GPU 结果的 50%。这个比例是当前基线，不是最终交付指标。

当前适配和推进重点

1. **推进结构迁移。**K3 的 Attention 基于千问的 Gated Data-Dependent Attention，并增加了可调节线性 Attention 的相关参数。由于 K2.5 已经有 KD/GDN 相关实现，适配时可以复用已有的状态更新、参数组织和部分算子路径，再根据 K3 的配置和权重布局补齐差异。
2. **接入推理框架执行链路。**需要完成模型注册、配置解析、权重映射、前向计算，以及 Prefill 和 Decode 两条路径的状态管理。Hybrid 结构下，Attention 的 KV Cache 与 KDA/GDN 的 SSM state、conv state 必须在同一个 token 边界上保持一致，否则会出现能加载但不能正确续推，或 Prefill 正确、Decode 结果漂移的问题。
3. **补齐部署和量化路径。**K3 参数规模较大，量化、TP/DP 组合、PD 分离以及状态传输都会直接影响显存占用和可部署性。当前不能只看模型能否跑通，还要验证长上下文、不同 batch、不同并行规模下是否会触发显存不足、通信等待或不支持的算子回退。

4. **建立精度与性能基线**。先用小 batch 和固定输入输出长度确认 checkpoint 加载、logits/生成结果和状态更新正确，再在代表性业务工况下测 TTFT、ITL/TPOT、吞吐、显存占用、通信时间和 MFU，形成后续优化的对照组。

“H20 的 50%”应该怎样理解

这个数字必须带测试口径。面试时应说明是**同一模型和精度下的阶段性端到端对标值**，通常对应某一组代表性负载的 tokens/s 或综合吞吐；它不等于所有输入长度、所有并行配置下都只有 H20 的一半，也不能直接当作理论算力利用率。更完整的记录至少应包含：

项目	需要固定或记录的条件
模型与精度	K3 具体规格、权重版本、量化方式、KV/state dtype
输入输出	prompt 长度、生成长度、是否 Chunked Prefill/MTP、是否包含多轮前缀
服务负载	并发数、请求到达分布、batch 上限、是否开启 Prefix Cache
并行策略	TP、DP、EP/CP、是否 PD 分离，以及 P/D 两侧的卡数
指标	TTFT、ITL/TPOT、tokens/s、尾延迟、HBM、通信占比、MFU

因此更稳妥的表述是：“K3 已经开始适配，当前功能链路在持续完善，阶段性性能约为同工况 H20 基线的 50%；后续还要通过算子、通信、量化和并行策略优化来提升。”

当前重点排查的性能瓶颈

- **KDA/GDN 状态更新路径**。如果状态更新、卷积状态维护或布局转换仍走通用实现，Decode 阶段容易受显存读写和 kernel launch 影响；需要确认是否存在算子回退、额外 transpose 或重复 state copy。
- **量化与权重搬运**。大模型 Decode 的权重读取占比高，反量化、非连续布局和低复用会放大带宽压力；应分别统计量化 kernel、GEMM 和权重搬运的时间。
- **Hybrid 调度和缓存**。Prefill/Decode、Attention/GDN 两类状态如果不能很好重叠，会出现 host gap、同步等待或状态传输开销；分块 Prefill 和 PD 交接还要检查边界是否引入额外拷贝。
- **跨卡通信**。TP/DP/PD 会引入集合通信；如果当前规格包含 MoE，还要单独观察 expert all-to-all 的重排、通信和负载不均。不能只看单个 GEMM 的 TFLOPS 判断整机性能。
- **形状和后端覆盖**。动态 batch、长短序列混合以及不规则 head 数可能导致专用 kernel 命中率低，进而出现小 kernel 过多或 GPU 空转。

这些是优先验证的方向，不代表 50% 已经被归因于某一个算子。正确做法是抓取算子输入输出维度、耗时、显存读写和通信时间，与理论性能及同类硬件结果对比，再用消融实验确认瓶颈。

下一步优化和可交付产出

1. 建立按算子和按阶段拆分的 profiling 表，区分 Prefill、Decode、通信、状态搬运和框架调度时间。

2. 优先优化占比最高且存在明显带宽或 launch 浪费的 KDA/GDN、投影、量化/反量化和通信算子；必要时先完成可验证的初版实现，再交给算子团队做深度调优。
3. 以客户的延迟、吞吐和显存约束为目标，组合调试 TP/DP/PD/CP、batch、Chunked Prefill 和 MTP 参数，避免用单一配置代表所有场景。
4. 每轮优化同时做精度回归、状态续推回归、长上下文回归和性能回归，最终沉淀为 K3 框架适配代码、量化/部署配置、算子优化需求清单和可复现的基线报告。

20. Continuous Batching 是怎么实现的？调度器每轮怎么组织 Decode 和 Prefill 请求？

直接回答：以执行迭代为边界动态调整 batch：完成的请求退出，新请求加入，未完成的继续。现代调度器通常以本轮 token 预算和缓存资源为约束，把请求当前待计算的 token 打包执行；开启 Chunked Prefill 时，一个 batch 可以同时包含 Decode token 和 Prefill chunk。[S3、S15、S26]

它与静态 batching 的区别

静态 batch 往往要等待整组请求完成再换下一组；continuous batching 在迭代之间回收已结束请求并补入新请求。无需等待同批最长输出结束。这里的“连续”是迭代级调度，不意味着 GPU kernel 执行到一半随时插入任意请求。[S26]

vLLM V1 的统一 token 视角

V1 scheduler 源码用以下概念描述请求进度：[S3]

```
num_computed_tokens：已经完成计算的 token 数
num_tokens_with_spec：prompt + 已有 output + speculative tokens 的总数
本轮待计算量：两者的差值，再施加预算和上下文等限制
```

所以单 token Decode 通常是待计算 1 个 token；一个未完成 Prefill 的请求可能还差几百个；speculative verify 可能要计算多个。**源码不依赖一个简单的“这一轮只能 Prefill 或 Decode”的全局相位。**[S3]

一轮调度的概念流程

以下为解释用的简化流程，省略异步调度、流式输入和部分后端细节：

```
1. 接收执行结果，更新请求进度；完成/取消请求释放引用。
2. 初始化本轮 token budget、request 数上限和 encoder budget。
3. 优先处理可运行的 running 请求：
   - 计算各请求当前待计算 token 数；
   - 受 token budget、chunk 限制、上下文长度等裁剪；
   - 检查/分配 KV 页与 Hybrid state；
   - 若媒体依赖未满足，先安排 encoder 或推迟相关 token；
   - 资源不足时按策略抢占或暂停。
4. 用剩余资源尝试接纳 waiting/恢复请求：
   - 做 prefix cache 匹配或等待远端缓存；
   - 为真正需要计算的后缀分配预算；
   - 满足依赖和资源条件后纳入本轮。
5. 输出各请求的 scheduled token 数、block table 和执行元数据。
6. Runner 打包执行；相应请求达到生成边界才采样，进入下一轮。
```

这是依据 V1 源码归纳的流程；真实调度还会因优先级、异步依赖、抢占与版本变化调整。[S3]

“Decode 优先”要说准确。Chunked Prefill 的文档描述优先安排 Decode，再把剩余 token budget 给 Prefill。V1 实现首先考虑 running 请求，而 running 中也可能包含尚未完成的 chunked prefill。不能把它简化为“所有代码都先遍历一个独立 Decode 队列，然后遍历 Prefill 队列”。[S3、S15]

一个具体的混合 batch

假设本轮 token budget 为 512，无其他限制：

```
64 个普通 Decode 请求，每个安排 1 token:  64
Prefill A 安排一个 256-token chunk:        256
Prefill B 安排一个 192-token chunk:        192
合计:                                     512
```

这是一个合法的概念打包示例，不保证默认 scheduler 在任意队列次序下正好生成这份配额。实际还受 max_num_seqs、KV 空间、encoder 预算、Mamba 边界对齐和公平性策略限制。

512 表示本轮待处理 token 数，不是 512 个请求，也不是所有请求历史长度之和。KV 空间仍要覆盖历史上下文；每个 Decode token 的 Attention 成本还随其上下文长度变化，所以固定 token budget 并不等于固定执行时长。

Runner 将这些 token 和请求元数据打包给模型，后端据此区分不同 query 长度与上下文范围；具体使用统一 kernel 还是分开的 Prefill/Decode 内核由 Attention backend 决定。Prefill A 若尚未处理到完整 prompt 尾部，本轮不会因此生成用户可见的输出 token。[S3、S15]

与 nano-vLLM 对照：第 1 题查看的 nano-vLLM 调度器一轮返回 Prefill batch 或 Decode batch，逻辑更简单；这正好体现了教学实现与完整 V1 调度器的差别。[S2、S3]

补充面试问答：Kimi/K3 并行、CP、MTP、性能分析与工程基础

第 18 题已经详细说明 K3 的适配状态和 H20 阶段性对标结果。本节把新增的并行策略、Context Parallel、实习产出、MTP 适配和性能分析问题集中整理，回答中区分了实际参与的工作和只做过代码 Review 或概念分析的部分。

Kimi 会采用怎样的并行策略？如果采用 PD 分离呢？

直接回答：当前主要支持对称的 PD 分离，例如 4P4D；在一个具体部署中，P 侧和 D 侧各配置相同数量的实例或并行组。硬件侧通常使用 32 卡完成 Kimi 的部署和量化，框架侧负责把 Prefill、Decode、状态交接和调度功能打通。即使不采用 PD 分离，也会结合 TP、DP 等策略。

- **P 侧（Prefill）：**处理输入上下文，建立 Attention KV Cache 以及 KDA/GDN 的递归状态，通常更看重矩阵计算效率、TTFT 和长 prompt 的处理能力。
- **D 侧（Decode）：**逐 token 生成，通常更容易受到权重读取、KV/state 访问、kernel launch 和并发调度影响，主要关注 ITL/TPOT 和持续吞吐。
- **对称 PD 的含义：**4P4D 表示 P、D 两侧采用对称的实例数量或资源组。每个实例内部还可能有 TP；因此 4P4D 不等于简单的 8 张卡，最终卡数取决于每个 P/D 实例的 TP、模型规模和部署方式。
- **状态交接：**P 完成到边界 `T` 后，需要把对应的 KV、SSM state、conv state 和位置/布局元数据交给 D。Hybrid 模型不能只传 KV，具体交接要求见第 7 题。

不做 PD 分离时，可以在同一组 GPU 上通过 TP 降低单请求延迟，通过 DP 承接更多独立请求；如果具体 Kimi 规格包含 MoE，还要根据框架支持情况考虑专家并行和 all-to-all。PD 是否划算取决于 P/D 负载比例、网络传输成本、排队情况和客户 SLO，不能因为卡数多就默认采用 PD。

TP 和 DP 哪种并行策略更优？如何选择？

直接回答：没有绝对更优的策略。TP 更偏向降低单请求延迟和加速 Prefill，DP 更偏向提升多请求整体吞吐；实际应在固定总卡数和客户业务指标下，对多组 TP/DP 组合做实测。

目标或约束	更可能优先增加的维度	原因和代价
低 TTFT、低单请求延迟	TP	将层内矩阵和 Attention heads 分到更多卡，单次计算并行度更高，但每层通信更频繁
Prefill 速度	TP	大矩阵乘更容易扩展，适合长 prompt；超过合适规模后通信和小矩阵效率会抵消收益
高并发、总吞吐	DP	多个副本分别接收请求，减少请求之间的相互影响，但每个副本都要保留模型权重
显存受限	先提高 TP，再检查复制开销	单卡权重压力通常下降，但 KV head、embedding、通信 buffer 等非完全切分部分可能产生冗余

目标或约束	更可能优先增加的维度	原因和代价
投机解码/MTP	结合 TP、draft 成本和接受率评估	更大的 TP 可能加速 target verify，但通信开销和 draft/verify 的资源比例也会改变

在固定总卡数 `G` 时，可以把候选配置写成 `G = TP × DP ×（其他并行维度）`，例如比较 `TP=8, DP=4` 与 `TP=4, DP=8`，同时记录 TTFT、ITL/TPOT、tokens/s、尾延迟、HBM 使用量、通信占比和投机解码接受率。TP 规模增大并不保证端到端线性加速：当 GEMM 已经接近通信或带宽上限时，继续扩大 TP 可能只增加 collective 次数。DP 规模增大也不一定持续提升吞吐，副本之间可能因为请求长度分布不同而负载不均。

面试时可以这样收束：“我会先按客户更看重延迟还是吞吐选择 TP/DP 的搜索范围，再固定模型、精度、上下文长度和并发做矩阵化测试，最后用实测结果决定配置。”

了解 Context Parallel（CP）吗？K3 能否使用 CP？

直接回答：CP 是沿上下文或序列维度切分的并行方式，与 TP、DP 正交，主要用于长文本，目标是分摊上下文相关的计算和缓存压力。K3 理论上可以使用 CP，但当前仍在迭代适配，不能把“理论可行”直接说成“所有版本已经稳定支持”。

- **和 TP 的区别：** TP 切层内权重、hidden 或 Attention heads；CP 切同一个请求的 token/context 范围。两者可以组合，前者解决单层计算和权重放置，后者解决长上下文的序列维度。
- **和 DP 的区别：** DP 把不同请求分给不同副本；CP 是把一个请求分到多张卡。因此 CP 不会像 DP 一样直接增加独立请求副本数。
- **适用场景：** 上下文很长、KV Cache 或 Attention 计算成为主要压力时更有价值。短 prompt 场景可能只增加通信和同步，收益不明显。
- **K3 的难点：** KDA/GDN 含有按 token 演进的递归状态，不能像普通全局 Attention 一样任意切块后独立计算；需要定义 chunk 顺序、状态传递或可结合的扫描方式。MLA/全注意力部分也需要明确每个 rank 如何获得计算所需的 K/V 信息。

此前做过相关代码 Review，并修复过 DeepSeek MLA 模型的 CP bug。这里应实事求是地说明：参与过 CP 逻辑检查和问题修复，但没有把团队底层 CP 通信模块全部独立实现。K3 是否开启 CP，仍要以具体框架版本、模型配置、上下文长度和通信后端的验证结果为准。

KDA 层和 MLA 层可以同时使用 CP 吗？CP 的实现方式有哪些？

直接回答：原则上可以，但两类层的切分和状态依赖不同，必须让它们在同一个逻辑 context 边界上协同。KDA 的递归状态、MLA 的 KV/latent 表示和跨卡通信不能各自采用互不兼容的边界。

在寒武实现中，当前可以采用一种伪 CP/VCP 思路：先把 Context 切分并分发到各卡，在进入 Attention 计算前再按计算需要重组。它能复用已有 Attention 接口，先完成功能验证，但是否真正降低了端到端内存和计算，要看重组发生的位置；如果很早就把完整 context 聚合回来，通信和内存收益会被抵消。

常见实现路径可以按数据交换方式理解：

1. **切分后重组（伪 CP/VCP）：**先分发 token，再在 Attention 前 all-gather 或重排，工程改动较小，但通信量可能较大。
2. **Ring/分块 Attention：**各卡按块循环交换 K/V 或中间结果，减少一次性聚合完整上下文的需求，但实现和调度更复杂。
3. **All-to-all/序列重排：**将序列或 head 重新分布到更适合计算的 rank，常与 TP 或特定 Attention 后端组合。
4. **分层或虚拟上下文并行：**在物理卡组和逻辑 context 分片之间再做映射，PCP、DCP 等名称属于具体实现的组织方式，不能脱离代码把缩写解释成唯一固定定义。

对 KDA 来说，还要处理“前一段 context 的 state 如何成为后一段的初始 state”；对 MLA 来说，要验证 latent/KV 的位置编码、head 映射和重组顺序。当前整体仍可归入 VCP/虚拟上下文并行的迭代方案，部分细节和性能收益需要继续完善。

CP 和 SP 一般有什么区别？开启 TP 时为什么经常搭配 SP？TP 的通信流程、横向切分和通信次数是什么？

直接回答：CP 和 SP 都涉及序列维度，但目的和作用域不同。CP 主要把一个长上下文请求跨设备切分；SP（Sequence Parallel）通常与 TP 配套，把 TP 组内的激活序列切开，降低 LayerNorm、Dropout、残差等部分的激活显存，并配合 collective 完成数据交换。

对底层 TP 通信，我掌握典型矩阵切分和 collective 逻辑，但没有直接负责团队底层通信模块的完整实现。面试时应明确这个边界，具体调用次数还要以实际代码和是否融合/重叠为准。

TP 的横向切分

- **Column Parallel Linear：**把权重的输出维度切给不同 rank。每个 rank 计算自己的输出列，通常不需要立刻做全量归约。
- **Row Parallel Linear：**把权重的输入维度切分，同时把对应的输入 hidden 分片送到各 rank；各 rank 得到部分输出后，需要 All-Reduce，或在开启 SP 时采用 Reduce-Scatter 形成序列分片。
- **Attention：**常按 head 或 head 相关维度切分 Q/K/V；输出投影通常是 Row Parallel，因此会在 Attention 输出处发生一次跨卡聚合。
- **MLP：**上投影/门控投影通常是 Column Parallel，下投影是 Row Parallel，因此也会产生一次跨卡聚合。

一个简化的 TP+SP 前向路径可以表示为：

```
序列分片 hidden
    ↓ All-Gather（进入需要完整序列的 Column Parallel 输入）
QKV / MLP 上投影：各 rank 本地计算
    ↓ Attention / 激活 / 门控：各 rank 本地计算
输出投影 / MLP 下投影：Row Parallel
    ↓ Reduce-Scatter（保留序列分片；无 SP 时通常是 All-Reduce）
残差与 LayerNorm：各 rank 处理自己的序列片段
```

典型实现中，一个 Transformer block 的 Attention 输出和 MLP 输出各有一组序列交换，因此大致可看到两组 Reduce-Scatter + 两组 All-Gather；如果关闭 SP，常见简化路径是两次 All-Reduce。实现可能通过通信融合、异步 overlap、缓存已聚合激活或不同 Attention backend 改变调用次数，所以不应把“每层固定四次”当成所有框架的硬规则。

为什么 TP 常搭配 SP：TP 把 hidden/head/FFN 维度切开，但 LayerNorm 和残差如果仍在每张卡保存完整序列，会产生激活显存冗余；SP 让序列维度也切开，把这部分内存压力降下来，同时利用必要的 All-Gather/Reduce-Scatter 保持计算语义。CP 更关注跨卡承接长上下文，SP 更像 TP 组内的激活布局配套。

回到实习经历，实际落地做了哪些工作?

直接回答：主要负责 bug 修复、新特性支持、模型适配，以及精度和性能测试分析；重点完成千问 3.5 的 MTP 适配与支持，也参与了千问、Kimi 等模型的测试。

可以按“问题—行动—产出”来讲：

工作类型	实际行动	可交付产出
Bug 修复	定位 Prefill、Decode、状态管理、分布式或后端兼容问题，构造最小复现并修复	代码 patch、回归用例、问题根因记录
新特性支持	参与 MTP、Prefill Chunk、PD/缓存等特性在目标硬件上的适配	功能实现、配置说明、精度/稳定性结果
模型适配	处理 Qwen3.5、Kimi 等模型的配置、权重、算子和状态差异	模型加载与推理链路、适配说明
性能分析	抓取算子输入输出、耗时、带宽和通信信息，定位瓶颈	profiling 表、优化建议、基线报告
算子协作	根据客户工况向算子团队提出优化需求，紧急情况下先完成初步实现	可验证的初版 kernel/路径、需求单和验收指标

这样表述能体现实际工程闭环，同时不会把团队协作完成的底层模块全部归为个人独立开发。

千问 3.5 的 MTP 做了哪些适配?

直接回答：主要是在 MTP Decode 阶段适配 GDN Backend。原生实现面向 NVIDIA 硬件，而自研硬件的算子输入、布局和状态存储方式不同，因此需要迁移输入输出约定、状态读写和验证路径；同时补齐原生实现不支持的 Memory State 存储形式。

适配时重点检查四类问题：

1. **输入输出布局：**确认 token、hidden、state、位置编码和 batch/sequence 维度在两套后端的顺序、dtype、stride 与对齐要求，不能只改一个 shape 字段。
2. **Memory State 生命周期：**明确 Prefill 产生的 state 如何进入 MTP Decode，draft 接受或拒绝后哪些 state 提交、回滚或复用；Memory State 需要和 KV Cache 对齐到同一个 token 边界。
3. **算子语义差异：**自研硬件可能要求不同的输入拆分、workspace 或状态布局，需要把 NVIDIA backend 的调用逻辑映射为目标硬件支持的算子组合。

4. **精度与接受率回归：**比较单步 Decode、MTP verify、接受/拒绝分支的 logits 和生成结果，并观察 token acceptance rate、吞吐和状态一致性，防止只验证“能跑”而遗漏回滚错误。

最终产出不只是把接口改到能编译，还包括后端适配代码、Memory State 管理路径、精度回归结果和 MTP 性能基线。

如果调整 MTP 参数来提升 token 接受率，可以调哪些参数？

直接回答：最直接的是减小 `draft_step`，也就是减少 Draft Model 一次生成的 token 数量。步数变小后，后面 token 受到前面预测误差累积的影响减弱，接受率通常会上升，但每次验证能覆盖的 token 也变少，端到端收益不一定更高。当前主要支持 Eager 模式，一次通常推测三四个 token。

可以把一次验证的有效收益粗略写成：

```
每轮期望接受 token ≈ draft_step × acceptance_rate
端到端收益还要扣除 draft 生成、verify、通信和调度开销
```

除 `draft_step` 外，只有在具体框架暴露相应开关时，才考虑 draft 模型采样参数、接受判定阈值、MTP head 数或最大验证长度；这些参数未必都能在当前实现中独立调节。温度、Top-p 等采样设置如果与目标模型不一致，可能改变接受率，但也会改变输出分布，不能只为提高接受率而随意修改。

建议在固定模型、输入长度和并发下对 `draft_step=2/3/4/5` 做小范围 sweep，同时记录接受率、每轮接受 token、TTFT、ITL、总吞吐和输出一致性。接受率提高但每轮接受 token 下降，或者 draft 成本占比变大时，整体性能仍可能变差。

如何评估算子是否还有优化空间、是否构成性能短板？

直接回答：先测出算子的实际耗时、输入输出规模和访存量，再与理论计算/带宽上限、同类 GPU 上的同形状算子以及多代硬件的代际变化做对比。单看 kernel 名称或单次 TFLOPS 不能证明它是瓶颈。

一个实用流程是：

1. **固定工况。**记录模型版本、dtype、batch、序列长度、TP/DP、warm-up 和正式迭代，避免比较不同 shape 得出错误结论。
2. **抓取算子数据。**获取输入输出维度、FLOPs、读写字节数、kernel 时间、调用次数、HBM/DDR 带宽、SM/Tensor Core 利用率和通信等待。
3. **建立理论上限。**计算受限算子看 `FLOPs / 峰值 FLOPs`，带宽受限算子看 `bytes / 带宽`；可用 Roofline 的 `max(计算下界, 访存下界)` 估计时间下限。
4. **横向对比。**参考 GPU 上同等 shape/dtype 的成熟实现，也比较新旧硬件的理论算力和带宽。新卡理论算力约为旧卡两倍时，实际只提升 1.1 倍并不一定是 kernel 错误，也可能已转为带宽、通信或调度受限。
5. **做消融确认。**对热点算子替换成参考实现、改变 block/layout、关闭通信重叠或调整 batch，观察端到端指标是否随之变化。

在 Kimi 2.6 性能分析中，重点是抓取算子信息、对比理论性能和实际运行性能，判断瓶颈落在计算、显存访问、跨卡通信还是框架调度，再向算子团队提出有输入规模、目标指标和验收方式的优化需求。最终应形成“证据—假设—修改—复测”的闭环，而不是只给出“这个算子慢”的结论。

MFU 和理论性能值如何计算？

直接回答：硬件团队提供峰值计算力、HBM/DDR 带宽、存储层级等基础参数；工具从算子输入输出和执行时间得到 FLOPs、字节数及实际吞吐，再用 Roofline 思路计算理论上限和 MFU。MFU 主要衡量计算利用率，带宽利用率需要单独报告。

对一个 GEMM，若矩阵形状为 `M×K` 与 `K×N`，可先估算：

```
FLOPs ≈ 2 × M × N × K                  # 再乘 batch 或 token 数
实际计算吞吐 = FLOPs / kernel_time
计算 MFU = 实际计算吞吐 / 硬件峰值 FLOPs
算术强度 AI = FLOPs / 实际读写字节数
理论时间下界 ≈ max(FLOPs / 峰值 FLOPs, bytes / 峰值带宽)
```

实际计算时还要考虑 dtype 对应的 Tensor Core 峰值、稀疏/量化规则、权重是否复用、cache 命中、padding、专家实际 token 数和通信时间。对于明显带宽受限的 GDN state 更新或 Decode 算子，即使 MFU 不高也可能已经接近带宽上限，此时应同时看 `实际带宽 / 峰值带宽`，不能单凭 MFU 判定还有同等比例的优化空间。

对端到端模型，通常还要拆分 Prefill、Decode、通信、状态搬运和调度时间；把所有时间简单相加后计算一个“模型 MFU”容易掩盖瓶颈。更可靠的报告至少同时给出算力利用率、带宽利用率、通信占比、TTFT/ITL 和吞吐。

千问 3.5 的 mamba-state 和普通 KV Cache 有什么区别？

直接回答：普通 KV Cache 保存 Attention 在每个历史位置产生的 Key/Value，随着上下文和生成长度持续增长；mamba-state 是线性注意力或状态空间路径在当前 token 边界上的递归状态，每一步会读入旧状态并更新成新状态，单个时间点的状态大小通常与序列长度无关。

可以从三个方面区分：

项目	普通 KV Cache	mamba-state（含 GDN/KDA 的递归状态）
表示内容	每个历史 token 的 K/V	历史 token 被压缩后的递归状态，通常还包括 conv state 等短历史
随序列长度变化	持续增长，通常近似与上下文长度成正比	单个边界的状态大小固定；若保存很多 Prefix checkpoint，总缓存量会随 checkpoint 数量增长
读取方式	Decode 时按当前 query 读取历史 K/V	Decode 时读取上一时刻 state，结合当前输入计算下一时刻 state 和输出

项目	普通 KV Cache	mamba-state（含 GDN/KDA 的递归状态）
Prefix 命中	命中 KV block 后可跳过对应 Attention Prefill	必须命中与 KV 相同 token 边界的 state，才能跳过对应递归计算

推理过程中，mamba-state 会在每一步生成后更新。Prefix Cache 若只保存最终 state，就无法自然恢复到任意较早的前缀；工程上通常每隔若干 token、在 block 或 Prefill chunk 边界保存一份 state。命中前缀时，框架复用最近的共同检查点，并从该位置继续计算未命中的后缀。保存粒度越细，命中后重算越少，但 state 快照占用的内存越多。

还需要注意边界一致性：KV Cache、SSM state、conv state 必须表示同一个“已经消费到的位置”。如果请求命中 1024 个 KV token，但 mamba-state 只保存到 768，就不能直接宣称 1024 个 token 全部命中，至少要从 768 继续计算 256 个 token。若启用了 ReplaySSM，尚未 materialize 的 replay 记录也属于逻辑 state，不能只保存 checkpoint tensor。

采用插件形式，还是修改原生 vLLM 框架？和华为昇腾的纯插件方案有什么差异？

直接回答：两种方式结合使用。一部分能力通过插件接入，另一部分涉及调度、缓存、模型执行和硬件后端的核心逻辑，需要复写或修改原生 vLLM。底层使用自研算子，调度模块对接 CNNL；原生 vLLM 的很多路径面向 NVIDIA 算子和 NCCL 调度模块，无法只靠外置插件直接适配，因此必须修改部分框架逻辑。

可以按层次理解：

- **插件适合的部分：**模型注册、特定算子封装、量化配置、后端选择、额外的输入输出适配，以及不改变调度不变量的功能扩展。
- **需要改框架的部分：**设备初始化、通信组、KV/mamba-state 内存管理、调度器资源判断、Model Runner 输入布局、CUDA Graph/Eager 分支和错误恢复等。只替换一个 Python layer，无法解决底层 collective 或状态生命周期差异。
- **硬件后端差异：**自研算子有自己的输入输出约定和布局，通信由 CNNL 等目标后端承接；社区原生路径常假设 NVIDIA kernel、CUDA stream 语义和 NCCL collective。即使 API 名称相似，也要验证同步、workspace、dtype、拓扑和异常行为。

因此与华为昇腾的纯插件方案相比，当前方案不是“全部 fork”也不是“全部插件”，而是按改动边界混合接入：稳定的上层接口尽量插件化，无法通过接口隔离的硬件特化路径直接改框架。本人主要接触模型相关模块、状态逻辑和算子适配，未深度参与 Model Runner 层的全部实现；面试时应明确这个边界。

内部 vLLM 版本为 0.25.1，为什么不能直接复用社区 vLLM 的 mamba-state 实现？

直接回答：并不是完全重写 mamba-state，而是需要在旧版本代码基础上做硬件特化和接口迁移。社区实现解决的是某一版本、某类硬件和某套内存布局下的问题，不能假设直接拷贝到内部 0.25.1 及自研硬件后仍然成立。

主要原因有三类：

1. **算子输入不兼容**。社区算子的输入定义、shape、stride、dtype 或 state layout 与目标硬件不一致，必须替换算子实现和调用适配。以 MTP verify 融合算子为例，早期可以先拆成多个小算子保证功能正确，再向算子团队申请高性能融合算子；拆分版是过渡实现，不应误认为最终性能方案。
2. **线性注意力的 state 管理尚不成熟**。PagedAttention 很适合按 block 管理逐 token 增长的 KV Cache，但 mamba-state 是递归状态，分块后不仅要存数据，还要定义检查点边界、state 复用、复制、回滚和生命周期。社区版本对线性注意力 state 的分块组织可能还不完善，不能直接套用 KV 的 page table 语义。
3. **硬件配置和块大小不同**。不同硬件的 block size、对齐、并行度、内存层级、通信后端和配置参数不同，需要做硬件特化；同时把可以泛化的修复整理后向上游社区提交 PR。

实际迁移顺序通常是：先对齐模型状态语义和精度结果，再替换目标硬件算子，最后围绕 block size、内存池、调度和通信做性能优化。这样可以把“版本/API 不兼容”“算子不兼容”和“算法状态设计不完整”分开定位。

简单写矩阵乘法，并说明矩阵分块优化 GEMM 的思路，以及如何让访存和计算互相掩盖

一个不考虑并行和边界优化的矩阵乘法为 `C = A × B`：

```
for i in [0, M):
    for j in [0, N):
        acc = 0
        for k in [0, K):
            acc += A[i, k] * B[k, j]
        C[i, j] = acc
```

直接按元素执行会反复从全局内存读取 A、B，数据复用差。分块 GEMM 把矩阵划分成 `BM×BK`、`BK×BN` 和 `BM×BN` 的 tile：每次先把 A、B 的 tile 搬到 shared memory、片上缓存或寄存器，再由一个线程块/计算单元反复复用，累加得到 C 的 tile，最后写回结果。这样可以减少重复 global memory 访问，提高算术强度。

典型思路是：

1. 让线程协同加载 A/B tile，并处理边界 padding 和对齐。
2. 在片上存储中循环 `K` 方向的 tile，让同一数据服务多个乘加。
3. 让每个线程或 warp 使用寄存器保存部分累加结果，减少中间写回。
4. 根据 Tensor Core、shared memory 容量、寄存器数量和矩阵形状选择 tile 大小。

要让访存和计算互相掩盖，可以使用双缓冲或多级流水：当前 tile 在寄存器/片上执行 MMA 时，异步拷贝指令同时把下一 tile 从 HBM 搬到 shared memory；下一轮计算开始前只需等待对应 buffer 就绪。概念上是：

```
时间 →   load(tile 0)   compute(tile 0)   compute(tile 1)   compute(tile 2)
                          load(tile 1)      load(tile 2)      load(tile 3)
```

当计算时间足够覆盖加载时间时，访存延迟就被计算隐藏；如果 tile 太小、带宽不足或计算单元不够忙，仍会出现 stall。实际还要用 occupancy、global load efficiency、shared memory bank conflict、Tensor Core utilization 等指标验证，不能只因为使用了分块就承诺固定加速比。

Linux 如何查看磁盘空间和可用空间？如何启动 Docker、进入指定容器？

直接回答：磁盘整体空间通常用 `df`，目录占用用 `du`；启动容器用 `docker run`，进入运行中的容器用 `docker exec`。 常用命令如下：

```
# 查看各挂载点总容量、已用、可用空间
df -h

# 查看当前目录下一级目录的占用，并按大小排序
du -sh ./* 2>/dev/null | sort -h

# 查看 Docker 镜像、容器和卷占用
docker system df

# 后台启动容器，命名并映射端口/目录
docker run -d --name llm-dev --gpus all -p 8000:8000 \
  -v "$PWD":/workspace -w /workspace IMAGE:TAG

# 查看运行中的容器
docker ps

# 进入指定运行中容器
docker exec -it llm-dev /bin/bash
# 镜像没有 bash 时使用
docker exec -it llm-dev /bin/sh
```

实际使用时要根据镜像、GPU runtime、端口和挂载目录调整参数。`docker start llm-dev` 用于重新启动已经创建但已停止的容器；它和首次用 `docker run` 创建容器不是一回事。日常开发可以把常用 Docker 操作封装成脚本；对不熟悉的 Linux 参数先查命令帮助或内部文档，再在非生产容器中验证。

线上 Docker 镜像内存持续泄漏，如何排查？查看显存占用的命令是什么？

直接回答：先区分“镜像/磁盘占用持续增长”和“容器内进程的主机内存持续增长”，再区分主机 RAM 与 GPU 显存。镜像层本身不会像进程一样发生运行时内存泄漏，线上通常是容器内服务、日志、缓存或 GPU 进程在增长。

建议按下面顺序排查：

1. **先确认增长对象和时间范围。**记录容器状态、重启次数、内存限制、进程 RSS、page cache、日志目录和 GPU 显存；同时固定流量、并发、输入长度和模型配置，避免负载变化被误判为泄漏。
2. **看容器与进程。**

   ```
   docker stats <container>
   docker inspect <container> --format '{{.HostConfig.Memory}} {{.HostConfig.MemorySwap}}'
   docker top <container>
   docker exec <container> sh -lc 'ps -eo pid,ppid,%mem,rss,vsz,cmd --sort=-rss | head'
   docker exec <container> sh -lc 'cat /proc/meminfo | head -20'
   ```

   容器 RSS 持续上升而请求量稳定，优先检查 Python/C++ 对象、线程、队列、连接、缓存、日志和 pinned memory；若 RSS 稳定但 cgroup 使用量增长，继续看 page cache、共享内存和文件句柄。

3. **看服务日志和句柄**。检查是否重复加载模型、请求异常未释放、队列积压、临时文件增长或日志没有轮转；必要时使用 `lsof -p PID`、heap profiler、Python tracemalloc 或目标语言的内存分析工具做快照对比。
4. **单独检查显存**。最常用命令是：

   ```
   nvidia-smi
   watch -n 1 nvidia-smi
   nvidia-smi --query-gpu=index,name,memory.total,memory.used,memory.free,utilization.gpu \
     --format=csv
   nvidia-smi pmon -s mu
   ```

   在容器内执行需要容器以 GPU runtime 启动且包含 `nvidia-smi`；也可以直接在宿主机观察。若显存持续增长，检查 KV Cache、mamba-state checkpoint、CUDA caching allocator、workspace、Pinned/H2D buffer、进程是否重复初始化，以及请求取消和异常路径是否释放状态。
5. **做最小复现和回归**。用固定 prompt、固定生成长度和固定并发运行一段时间，分别关闭 Prefix Cache、MTP、量化、CUDA Graph、PD/通信缓存等开关，找出导致增长的最小组合。确认是框架缓存的正常高水位还是不可回收的活跃分配，再决定调整缓存上限、修复生命周期还是重启服务。

线上处理还要设置 cgroup/Docker memory limit、显存水位告警、请求超时和优雅重启，防止排查期间拖垮同机其他服务；这些是隔离和止损措施，不能替代最终的泄漏根因修复。

AI Infra 高频 100 问：腾讯混元优先

本节是面向大模型推理、训练基础设施和多模态服务岗位的高频问答清单，优先围绕腾讯混元开源模型与官方部署材料整理，再扩展到 vLLM、SGLang、TensorRT-LLM、NVIDIA/PyTorch 等通用 AI Infra 知识。混元相关数字和命令以具体模型仓库版本为准；不同版本、硬件和后端不能直接横向比较。[S27–S43]

A. 腾讯混元模型与部署重点（1–25）

1. Hunyuan-A13B 是什么模型?

答： Hunyuan-A13B 是腾讯混元开源的 MoE 大语言模型，总参数约 80B，每个 token 激活约 13B 参数。它采用细粒度专家结构，目标是在保持较大模型容量的同时降低单 token 计算量；部署时仍要为总权重、KV Cache、通信 buffer 和未激活专家权重预留显存。[S27]

2. “80B 总参数、13B 激活参数”分别影响什么?

答： 总参数主要影响权重存储和加载成本，激活参数主要影响每个 token 的计算量。MoE 推理并不意味着只需要存 13B 参数，所有专家权重通常仍需可访问；因此显存规划要同时考虑总权重、量化精度、TP/EP 分片和 KV Cache。[S27]

3. Hunyuan-A13B 的长上下文和 GQA 对推理有什么意义?

答： 官方仓库说明 A13B 原生支持 256K 上下文，并采用 GQA 以降低 KV Cache 压力。长上下文场景的主要风险从“模型权重能否放下”转向 KV Cache 容量、Attention 访存、调度和尾延迟；GQA 能减少 KV head 数，但不能消除随序列增长的 KV 存储。[S27]

4. Hunyuan-A13B 的 FP8 和 Int4 量化有什么区别?

答： 官方材料中的 FP8 是静态量化，需要用校准数据预先确定 scale，并同时压缩权重/激活相关路径；Int4 采用 GPTQ W4A16，主要压缩权重，激活通常保持 FP16/BF16。FP8 往往更利于 Tensor Core 吞吐，W4A16 更节省权重显存，但要关注反量化、kernel 支持和精度回归。[S27]

5. Hunyuan-A13B 支持哪些推理后端?

答： 官方仓库给出了 TensorRT-LLM、vLLM 和 SGLang 的部署路径，并提供对应容器或启动示例。选择后端时要比较模型注册、MoE dispatch、量化 kernel、PagedAttention、通信后端和多机支持，而不是只看是否能加载模型。[S27]

6. 为什么要关注 A13B 官方 vLLM 容器的版本和 CUDA 要求?

答： 官方示例明确给出包含 vLLM 0.8.5 的镜像，并注明该镜像需要 CUDA 12.8；这是该发布材料对应的版本口径，不应泛化为所有 vLLM 版本。部署时应固定镜像、驱动、CUDA、PyTorch、vLLM 和模型代码版本，否则自定义模型代码或算子 ABI 变化可能导致加载失败或性能变化。[S27]

7. A13B 用 vLLM 或 SGLang 部署时首先检查哪些项?

答：先确认模型类注册、`trust_remote_code` 需求、TP/EP 参数、量化格式、最大上下文和 GPU 数量；再做单卡/小 batch 精度回归，最后验证多卡通信和吞吐。官方启动示例使用 TP=4，但这只是示例配置，实际 TP 要根据模型规格、GPU 显存和目标并发重新搜索。[S27]

8. Hunyuan-Large 的规模和激活参数是多少?

答：官方仓库将 Hunyuan-Large 描述为约 389B 总参数、52B 激活参数的 Transformer MoE。它的部署重点不只是算力，还包括专家权重分片、跨卡 token dispatch、长上下文 KV 管理和多机通信。[S28]

9. Hunyuan-Large 的 CLA 解决什么问题?

答：CLA（Cross-Layer Attention）用于减少跨层重复保存的 KV 表示，从而降低 KV Cache 的容量和访存压力。官方材料称其可显著减少 KV Cache 占用；面试时应说明具体收益依赖实现、序列长度、层配置和 dtype，不能把宣传比例当成所有负载的实测结果。[S28]

10. Hunyuan-Large 的 256K/128K 上下文口径如何理解?

答：官方材料区分了预训练模型和 Instruct 模型：预训练模型支持到 256K，Instruct 模型支持到 128K。部署时要以实际 checkpoint 的 config、位置编码实现和服务参数为准；最大上下文并不等于在目标 GPU 上能以任意 batch 稳定运行。[S28]

11. Hunyuan-Large 为什么需要较大的 GPU 集群?

答：官方部署材料给出了 BF16 在 H20 上至少 16 GPU 的配置示例，原因包括总权重规模、专家分片、KV Cache、workspace 和通信 buffer。实际卡数还取决于 FP8/Int8、TP/EP 组合、最大上下文、batch 和是否启用 CPU offload。[S28]

12. 如何看待 Hunyuan-Large 官方 vLLM/TRT-LLM 性能数据?

答：官方 README 给出了特定 H20、输入长度和 batch 条件下的 tokens/s 对比，并报告了 FP8 和 TRT-LLM 的收益。复述数据时必须带上硬件、模型精度、输入输出长度、batch、并行配置和版本；这些数字是基准条件下的结果，不能直接当作任意生产流量的保证。[S28]

13. HunyuanImage-3.0 的架构和普通文生图模型有什么不同?

答：官方材料将它描述为统一的自回归多模态框架，而不是单纯的 DiT 文生图流水线。文本 token 使用自回归建模，图像生成路径结合 VAE latent 和扩散式预测，并通过视觉编码器、VAE 和 projector 把多种模态对齐到统一表示空间。[S30]

14. HunyuanImage-3.0 的 MoE 配置有什么特点?

答：官方 README 说明其包含 64 个专家，每 token 激活 8 个专家，并带一个 shared MLP；总参数约 80B，激活参数约 13B。推理时要同时考虑专家路由、FlashInfer/Grouped GEMM、跨卡通信以及图像 token 数量。[S30]

15. HunyuanImage-3.0 为什么推荐 FlashInfer/FlashAttention?

答：官方仓库提供 FlashInfer、FlashAttention 等加速选项，并说明首次运行可能需要较长 kernel 编译时间。FlashInfer 重点优化推理阶段的 Attention/MoE 等不规则路径，实际收益依赖 GPU、shape、编译缓存和 batch；首次编译时间不能计入稳态吞吐比较。[S30]

16. HunyuanVideo 的总体推理链路是什么?

答：HunyuanVideo 先用 Causal 3D VAE 将图像/视频压缩到时空 latent，再由文本编码器提供条件，DiT 在 latent 空间中迭代生成，最后由 VAE 解码回视频。其架构采用文本和视频 token 的 dual-stream 到 single-stream 设计，先分别处理，再在后续 Transformer 中融合。[S29]

17. HunyuanVideo 的显存和 FP8 优化如何理解?

答：官方 README 给出了 720p、129 帧等配置的显存需求，并提供 FP8 权重以降低显存。FP8 主要降低权重和相关计算的存储压力；如果瓶颈是激活、Attention 中间结果、VAE 或通信，单独量化权重不会等比例降低峰值显存。[S29]

18. HunyuanVideo 的多卡并行采用什么思路?

答：官方代码通过 xDiT 和 Unified Sequence Parallelism（USP）进行多 GPU 推理，并提供 Ulysses degree、Ring degree 等组合。Ulysses 更偏序列/Attention 维度的重排，Ring 更偏块状循环通信，实际组合必须满足视频尺寸、帧数和进程数的约束。[S29]

19. HunyuanVideo 1.5 的主要 AI Infra 特点是什么?

答：官方技术报告将其描述为约 8.3B 参数的两阶段视频生成系统：第一阶段用 DiT 生成 480p/720p 视频，第二阶段用视频超分网络提升到 1080p；其 DiT 使用 selective and sliding tile attention（SSTA）降低长视频 token 的 Attention 开销。部署时要把 DiT、VAE、超分和 offload 分开测量。[S31]

20. HunyuanVideo 1.5 的 cache/compile 优化如何理解?

答：官方报告提到使用 SageAttention、`torch.compile` 以及 diffusion sampling 中的 feature caching 来减少重复计算。feature cache 需要验证相邻 diffusion step 的特征近似误差，不能把它和 LLM 的 KV Cache 混为一谈；质量、步数、分辨率和视频长度都会影响收益。[S31]

21. 腾讯混元多模态服务为什么需要 processor cache 和 encoder cache?

答：Processor cache 保存图片解码、Resize、Normalize、抽帧等 CPU 预处理结果；encoder cache 保存 ViT/视觉编码器输出。前者减少 CPU 处理时间，后者减少 GPU 编码时间，但 cache key 必须包含媒体内容、预处理参数、模型版本和分辨率，不能只用 `<image>` 占位符。[S29、S30]

22. 如何为不同业务选择 Hunyuan 模型?

答：文本长上下文和通用对话优先比较 A13B 与 Hunyuan-Large 的质量、显存和吞吐；图像理解/生成关注 HunyuanImage-3.0 的统一多模态链路；视频生成则关注 HunyuanVideo/1.5 的 latent 压缩、帧数、VAE 和 DiT 并行。最终以业务 SLO、质量指标、GPU 预算和可维护性做选择，而不是只按参数量排序。[S27–S30]

23. 把 Hunyuan 适配到自研硬件时最难的是什么?

答：难点通常包括模型权重布局、Attention/MoE/GEMM 算子、量化格式、通信 collective、KV/state 管理和动态 shape。应先保证单卡精度和状态正确，再替换硬件算子，最后优化 TP/EP/CP、通信重叠和内存布局；不要一开始就同时改模型、调度和通信。

24. 如何为 Hunyuan 建立可信的性能基线?

答：固定模型 checkpoint、dtype、输入输出长度、并发、TP/EP、后端版本和 warm-up，分别测 TTFT、ITL/TPOT、吞吐、尾延迟、显存、通信占比和 MFU。对 MoE 还要记录每个专家 token 分布，对多模态还要拆分 processor、encoder、LLM/DiT、VAE 和 H2D 时间。

25. Hunyuan 开源代码部署时有哪些安全和版本风险?

答：某些 Transformers/vLLM 部署示例需要 `trust_remote_code`，它会执行模型仓库中的 Python 代码，应固定可信 commit、审查代码并隔离运行环境。还要锁定 CUDA、驱动、PyTorch、后端、模型 config 和量化工具版本，避免“同一模型名但代码路径不同”造成不可复现。[S27、S28]

B. 推理框架与模型执行（26–45）

26. 一个现代 Decoder-only Transformer Block 由什么组成?

答：典型顺序是 `RMSNorm → Attention/MLA/GDN → Residual Add → RMSNorm → SwiGLU Dense/MoE FFN → Residual Add`。不同模型可能加入 QK-Norm、门控残差、局部 Attention 或 Mamba，但核心仍是“跨 token 交互”和“逐 token 特征变换”两类子层。

27. MHA、MQA、GQA 和 MLA 如何影响推理?

答：MHA 为每个 Query head 配独立 K/V，表达能力强但 KV Cache 大；MQA 共享一组 K/V，缓存最小；GQA 在两者之间折中。MLA 进一步压缩 latent KV 表示，减少长上下文的缓存和访存，但需要专门的权重、位置编码和 Attention backend。

28. GDN/Mamba 与普通 Attention 的缓存有什么不同?

答：普通 Attention 主要缓存每个历史位置的 K/V，长度随上下文增长；GDN/Mamba 维护递归 state，单个时间边界的 state 大小通常固定，但每层可能还有 conv state。Hybrid 模型的 Prefix Cache、PD 传输和投机回滚必须同时管理 KV、SSM state、conv state 和位置边界。

29. KV Cache 大小如何粗略估算?

答：单层 KV Cache 的逻辑元素数可粗略写成 `2 × sequence_length × num_kv_heads × head_dim`，再乘 batch、层数和每元素字节数。GQA/MQA 通过减少 `num_kv_heads` 降低容量；PagedAttention 减少碎片，但不会消除实际 KV 数据量。

30. Prefix Cache 解决什么问题?

答：它缓存已经计算过的公共前缀对应的 KV 或 Hybrid state，后续请求命中时跳过重复 Prefill。缓存 key 必须包含 token 内容、媒体内容、模型/LoRA 版本、位置相关配置和状态布局；对 GDN/Mamba，还要保存与 KV 同一边界的 state。

31. PagedAttention 的核心思想是什么?

答：它把 KV Cache 切成固定大小的 block/page，用 block table 将逻辑序列映射到非连续物理块，从而减少连续大块分配带来的碎片和预留浪费。代价是多一次页表寻址和更复杂的后端访问，block size 需要结合上下文、并发和 kernel 访问模式选择。

32. Continuous Batching 如何工作?

答：调度器以迭代为边界移除完成请求、接收新请求，并按 token budget 把 Decode token 和 Prefill token 打包。它不会在一个 CUDA kernel 中途插入请求；真实实现还要同时检查 KV 空间、encoder budget、优先级、抢占和状态资源。

33. Chunked Prefill 解决什么问题?

答：它把长 Prompt 拆成多个 chunk，避免一次性 Prefill 长时间占用 GPU，给 Decode 请求持续获得执行机会。chunk 太大仍会阻塞 Decode，太小则导致更多调度和 kernel launch、较小 GEMM 以及更差 Prefill 效率，因此要用 TTFT/ITL/吞吐联合搜索。

34. PD 分离为什么要传递 Cache?

答：Prefill 节点完成 Prompt 后，Decode 节点必须拿到可继续执行的完整状态。普通模型至少传各层 KV；Hybrid 模型还要传 GDN/SSM state、conv state、位置、dtype/layout、block table 和 rank 映射，且所有状态必须对应同一 token 边界。

35. MTP/Speculative Decoding 的推理流程是什么?

答：Draft 模型或 MTP head 先提出多个候选 token，Target 模型并行验证，再提交 accepted prefix、回滚 rejected suffix。性能取决于接受 token 数、draft 成本、verify kernel、state 回滚成本和调度开销，不能只看 acceptance rate。

36. token acceptance rate 如何影响投机解码收益?

答：粗略期望值可写为 `draft_step × acceptance_rate`，但端到端还要扣除 draft、verify、通信和状态管理时间。减小 draft step 通常可能提高接受率，却也减少每轮可接受的最大 token 数，需要在固定负载下测吞吐、ITL 和每轮接受 token。

37. Scheduler 的 token budget 是什么?

答：它是本轮最多安排的待计算 token 数，不是请求数，也不是所有历史 token 数。例如 budget=512 时，可能是 64 个 Decode token 加两个 Prefill chunk。预算还要受到最大序列数、KV 空间、encoder 预算、状态对齐和公平性策略约束。

38. CUDA Graph 主要优化什么?

答: CUDA Graph 将重复执行结构捕获为图,降低 CPU 逐 kernel launch 和调度开销,特别适合形状稳定、短 kernel 多的 Decode。它不会自动减少 FLOPs,也不会自动把多个 kernel 融合;动态 shape、padding 和 graph memory 可能抵消收益。

39. `torch.compile` 和算子融合有什么关系?

答: `torch.compile` 是图捕获、变换和代码生成工具链,算子融合只是它可能执行的一种优化。编译器还可能做布局、内存规划、常量传播和库调用;手写 CUDA/Triton 也可以独立实现融合,不需要编译整个模型。

40. FlashAttention 为什么能降低 Attention IO?

答: 它用 tile 化方式在片上存储中完成部分 QK、Softmax 和 V 计算,避免显式物化完整的 Attention score/probability 矩阵。收益依赖序列长度、head shape、dtype、后端和 mask 类型;不是所有形状都能达到相同加速。

41. FlashInfer 主要解决什么问题?

答: FlashInfer 面向推理场景的 Attention、KV Cache、Paged Attention 和部分 MoE/不规则 shape 路径,重点是减少 Python/launch 开销并提供针对推理布局的高性能 kernel。使用时要关注编译缓存、CUDA/编译器版本、支持的 dtype 和模型 shape。[S30]

42. Dense、MoE 和 Hybrid 模型的执行路径有什么差异?

答: Dense 模型每个 token 经过同一组 FFN 权重;MoE 需要 Router、Dispatch、专家 GEMM、Combine 和可能的 All-to-All;Hybrid 模型还要在 Attention KV 与递归 state 之间维护不同缓存。性能分析必须分开统计这些路径,不能只用 Dense GEMM 的指标代表全模型。

43. 权重量化和 KV Cache 量化分别解决什么问题?

答: 权重量化降低模型权重显存和权重读取带宽,常见 FP8、INT8、GPTQ W4A16;KV Cache 量化降低随上下文增长的缓存容量和访问量。两者的 scale、误差、反量化 kernel 和硬件支持不同,应分别做精度与性能回归。

44. FP8 静态量化的 scale 为什么重要?

答: FP8 可表示范围和精度有限,需要用校准数据估计权重/激活的 scale,避免少量 outlier 导致溢出或有效精度损失。校准集应覆盖真实请求分布;只在短文本或单一 batch 上校准,可能在长上下文、MoE 热点或多模态输入上出现误差。

45. 量化后为什么不一定变快?

答: 如果硬件没有高效量化 GEMM,反量化、scale 读取和布局转换可能抵消权重节省;小 batch 还可能受 kernel launch 和权重搬运影响。要分别测量量化 GEMM、反量化、显存带宽、端到端 TTFT/ITL,而不是只看模型文件变小。

C. 并行、通信与分布式执行（46–65）

46. Tensor Parallel（TP）切分什么?

答： TP 在层内切分权重或 Attention head，使一个 token 的矩阵计算跨多个 GPU 执行。Column Parallel 通常切输出维度，Row Parallel 切输入维度并在结果处做 All-Reduce 或 Reduce-Scatter；TP 越大，单请求计算可能更快，但 collective 次数和通信敏感性也增加。

47. Data Parallel（DP）适合什么场景?

答： DP 为不同 GPU 副本分配不同请求，主要提高并发吞吐和隔离请求之间的干扰。代价是每个副本要保存模型权重，显存复制增加；如果请求量不足或长度分布不均，DP 副本可能空闲。

48. Pipeline Parallel（PP）解决什么问题?

答： PP 把不同 Transformer 层放到不同设备或 stage，主要解决单卡/单组无法容纳完整模型的问题。它会引入 pipeline bubble、micro-batch 调度和跨 stage 激活传输，单请求低延迟场景通常需要谨慎评估。

49. Expert Parallel（EP）如何工作?

答： EP 将不同 MoE experts 放到不同 rank，token 经过 Router 后通过 All-to-All 发送到专家所在 rank，再把结果发回。EP 能减少单卡专家权重压力，但通信量、负载不均和专家热点会成为核心瓶颈。

50. TP 和 EP 为什么可以同时使用?

答： TP 切每个专家内部的矩阵，EP 切专家集合的放置，因此是不同维度的并行。一个 token 可能先通过 EP 到目标专家，再在专家内部执行 TP GEMM；实际 rank 组和通信顺序必须避免重复搬运和拓扑冲突。

51. Context Parallel（CP）解决什么问题?

答： CP 沿序列/context 维度切分一个长请求，主要分摊长上下文 Attention、KV 或相关状态压力。它与 TP、DP 正交，但需要处理位置编码、跨卡 K/V 交换、GDN state 顺序和 chunk 边界，短文本场景可能被通信开销抵消。

52. Sequence Parallel（SP）和 CP 有什么区别?

答： SP 常作为 TP 组内的激活布局配套，把 LayerNorm、残差和部分激活沿序列切开，以降低中间激活显存；CP 更关注把一个长上下文请求跨设备承接。两者都涉及序列维度，但作用范围、通信位置和目标不同。

53. Ulysses、Ring 和 USP 适合什么场景?

答： Ulysses 常通过序列/head 重排实现 Attention 并行，Ring 通过分块循环交换 K/V 或中间结果，USP 将这些序列并行方式统一成可组合接口。HunyuanVideo 官方多卡推理使用 xDiT/USP，并提供 Ulysses degree 与 Ring degree 组合，配置要服从视频尺寸和帧数约束。[S29]

54. All-Reduce、All-Gather、Reduce-Scatter、All-to-All 有什么区别?

答： All-Reduce 让所有 rank 得到归约结果；All-Gather 聚合各 rank 的分片；Reduce-Scatter 先归约再分片；All-to-All 将每个 rank 的不同分片发送给所有目标 rank。TP 常见 All-Reduce/Reduce-Scatter，EP 常见 All-to-All，具体次数会因融合和重叠而变化。

55. 什么是通信计算重叠?

答： 它让通信流传输下一批数据的同时，计算流处理上一批已到达数据，例如 All-to-All chunk 与专家 GEMM pipeline。有效重叠需要异步 collective、事件依赖、双缓冲和足够大的 chunk；必须用时间线确认通信和计算确实并发。

56. 异步调度与连续批处理有什么区别?

答： 连续批处理决定每轮有哪些请求，异步调度决定 CPU 准备、GPU 执行、通信和结果处理是否可以并行推进。异步调度可以提前准备下一轮 batch，但如果存在同步点、数据依赖或后端不支持非阻塞操作，实际收益会很小。

57. 如何识别 MoE 专家负载不均?

答： 记录每轮每个专家收到的 token 数、最大/平均比值、P50/P99 专家耗时和 All-to-All 字节量。若少数专家长期成为热点，Grouped GEMM 仍会被最大 `M_e` 和尾部专家拖慢，应考虑路由约束、专家复制、重分布或容量控制。

58. MoE Grouped GEMM 为什么比逐专家 GEMM 更好?

答： 逐专家调用会产生大量小 GEMM 和 launch，Grouped GEMM 将不同专家的矩阵乘作为一组任务调度，通常更好利用 Tensor Core。它不能自动消除 token 分布极端不均，也不能替代 dispatch/combine 和通信优化。

59. NCCL、CNNL 和自研通信后端如何选择?

答： NCCL 主要面向 NVIDIA GPU 集合通信，CNNL 或其他厂商库承担对应硬件的通信和算子接口。迁移时不能只替换函数名，还要验证 stream 语义、拓扑、dtype、同步、错误处理和 collective 算法；上层 vLLM/SGLang 调度逻辑也可能需要硬件特化。

60. 多机推理为什么要关注拓扑?

答： 同机 NVLink、PCIe、跨机 RoCE/InfiniBand 的带宽和延迟差别很大，TP/EP/PP 的 rank 放置应尽量匹配通信模式。把高频 TP collective 跨到慢链路，可能使增加 GPU 后反而变慢；部署前要验证 NCCL/CNNL 拓扑和实际带宽。

61. Rank mapping 为什么会影响性能?

答： rank mapping 决定模型 shard、专家、通信组与物理 GPU/网卡的对应关系。错误映射可能让同一 TP 组跨 NUMA、跨交换机或跨慢链路，造成通信尾延迟和负载不均；应把逻辑 rank、GPU、NIC 和拓扑记录在部署配置中。

62. PD 分离必须对称吗?

答：不必须。对称的 4P4D 简单、容易验证；如果 Prompt 很长而输出较短，可以增加 P 资源，如果输出很长或并发高，可以增加 D 资源。非对称方案需要评估模型副本、状态传输、排队和 P/D 负载平衡，不能只按卡数比例决定。

63. 分布式加载权重如何避免启动瓶颈?

答：使用分片 checkpoint、并行读取、节点本地缓存和 rank-aware 加载，避免每个 rank 从网络重复拉取完整权重。还要区分文件读取、反序列化、权重转换、GPU H2D 和 NCCL 初始化时间，否则启动慢的根因不清楚。

64. CPU offload 和 GPU memory paging 适合什么场景?

答：它们可以降低 GPU 峰值显存，让较大模型或视频 pipeline 在较小 GPU 上运行，但会引入 PCIe/互联传输和同步。只有当被 offload 的模块访问频率低、传输能与计算重叠或业务能接受更高延迟时，才可能划算。[S29]

65. 多 GPU 服务如何处理单卡故障或通信失败?

答：生产系统需要健康检查、请求超时、进程组重建、实例摘除和可恢复的模型加载流程。对正在进行的 Prefill/Decode，要定义状态是否丢弃、是否回源重算，以及客户端如何重试；不能只依赖底层 collective 返回错误。

D. 算子、性能和 Profiling（66–82）

66. GEMM 是什么?

答：GEMM 是 `C = αAB + βC` 的通用矩阵乘法，大模型中的 QKV 投影、FFN、LM Head、MoE Expert 都大量使用它。性能取决于 `M/N/K`、dtype、Tensor Core、tile、内存布局和权重读取，而不是只看理论 FLOPs。

67. Roofline 如何判断算子瓶颈?

答：先估算 `FLOPs` 和实际读写字节数，得到算术强度 `AI=FLOPs/bytes`，再比较计算峰值和带宽峰值。理论时间下界可以粗略写成 `max(FLOPs/峰值算力, bytes/峰值带宽)`；实际还要扣除 launch、同步、通信和 cache 命中差异。

68. MFU 如何计算?

答：算子级计算 MFU 可写成 `实际 FLOPs/执行时间 ÷ 硬件峰值 FLOPs`。如果算子主要受带宽限制，MFU 低并不说明一定还有同等比例的计算优化空间，还要报告实际带宽/峰值带宽、通信占比和端到端 TTFT/吞吐。

69. Nsight Systems 和 Nsight Compute 分别看什么?

答：Nsight Systems 看全局时间线：CPU、GPU kernel、H2D、通信、空洞和线程调度；Nsight Compute 深入单个 kernel：占用率、Tensor Core、内存吞吐、寄存器、shared memory 和 stall。通常先用 Systems 定位时间段，再用 Compute 分析热点 kernel。

70. 如何区分计算瓶颈和显存带宽瓶颈?

答：计算瓶颈通常表现为 Tensor Core/SM 利用率高、带宽未到峰值；带宽瓶颈表现为 DRAM/HBM 高、算术强度低、计算单元空闲。小矩阵还可能是 launch 或并行度瓶颈，需要结合 kernel 数、空洞和 shape 分布判断。

71. 什么是 host gap?

答：host gap 是 GPU kernel 之间由于 CPU 调度、Python、Dispatcher、同步或数据准备产生的空隙。算子融合、CUDA Graph、异步预取和减少 Python 调用都可能改善它，但网络等待、显存不足和后端同步不会仅靠融合自动消失。

72. Occupancy 为什么不是越高越好?

答：Occupancy 表示每个 SM 同时驻留的线程/warp 比例，高 occupancy 有助于隐藏延迟，但寄存器、shared memory 和 Tensor Core 资源竞争可能使过高 occupancy 牺牲单个 warp 的效率。应结合吞吐、stall、寄存器 spill 和实际 kernel 时间判断。

73. GEMM 的分块和流水线如何隐藏访存?

答：将 A/B 切成 tile，协同加载到 shared memory，再用寄存器/Tensor Core 重复计算；双缓冲让当前 tile 计算时异步加载下一 tile。理想情况下加载时间被计算覆盖，但 tile、寄存器、shared memory 和边界 padding 需要结合 shape 调优。

74. Layout 为什么会影响算子性能?

答：连续、对齐且符合 kernel 访问方向的布局能减少非合并访存和 transpose；不合适的 `[B,T,H,D]` 与 `[T,B,H,D]` 转换可能产生额外 kernel 和显存流量。量化、GQA、MoE token permutation 还会增加 layout 约束，不能只看逻辑 shape。

75. 量化 GEMM 的性能瓶颈有哪些?

答：包括权重解包、scale 读取、反量化、低比特 Tensor Core 指令、非连续权重布局和小矩阵效率。分析时应拆分 quantized GEMM、dequant、pack/unpack 和 memory copy，确认权重节省是否真正转化为端到端收益。

76. 为什么 Decode 的 GEMM 经常不如 Prefill 高效?

答：Prefill 的 `M` 是多个 token，矩阵乘并行度和权重复用较好；Decode 常接近 `M=1` 或小 batch，权重读取占比高，Tensor Core 不易填满。增大 batch、使用连续批处理、量化和专门的 decode kernel 都可能改善，但会影响延迟。

77. Attention 的 IO 开销主要来自哪里?

答：长上下文 Decode 主要来自读取历史 KV；Prefill 还要处理 Q/K/V 投影、score、softmax 和输出。PagedAttention、GQA/MLA、FlashAttention、KV 量化和更好的 block layout 可以降低不同环节的压力，但需按上下文长度和 batch 实测。

78. GDN state 更新为什么可能成为瓶颈?

答：递归 state 需要每步读取、更新和写回，若 state 矩阵较大，Decode 可能受显存带宽和同步依赖限制；conv state、layout 转换和 Prefix checkpoint 也会增加流量。可以检查 state dtype、读写次数、融合程度、ReplaySSM 和缓存边界。

79. 动态 shape 如何影响性能?

答：batch、序列长度、图像分辨率和专家 token 数变化会导致 kernel 选择、CUDA Graph、编译缓存和 GEMM tile 命中率变化。常见做法是 shape bucketing、padding、动态 kernel 和 eager fallback，但 padding 过多会浪费 FLOPs 和显存。

80. CUDA Graph 的内存代价是什么?

答：Graph 通常需要稳定的输入输出地址和可复用 buffer，并可能为不同 batch/shape 保存多张图。图越多、bucket 越细，捕获和显存开销越大；应统计 graph memory、padding、capture time 和命中率，而不是只看 replay 时间。

81. 如何设计可信的性能测试?

答：先 warm-up，排除首次编译、权重加载、CUDA Graph capture 和 cache 冷启动，再固定输入输出长度、并发、随机种子和版本。至少报告平均值、P50/P95/P99、TTFT、ITL、吞吐、显存和错误率，并提供多轮重复的方差。

82. 为什么尾延迟比平均吞吐更重要?

答：在线服务通常有 TTFT、TPOT/ITL 和 P99 SLO，少数长 Prefill、专家热点、通信抖动或 GC 就可能拉高尾延迟。优化时不能只追求平均 tokens/s，还要观察排队时间、batch 形成时间、P/D 不平衡和最慢 rank。

E. 服务工程、缓存与稳定性（83–100）

83. 如何设计 Hunyuan 的 OpenAI 兼容服务?

答：API 层负责请求校验、鉴权、流式输出、超时和错误码；推理层负责 tokenizer、scheduler、模型执行和状态管理。要明确 chat template、reasoning 输出、图片/视频字段、最大输入、取消请求和 usage 统计，避免只做一个能返回文本的 Demo。

84. 为什么需要 admission control?

答：当 KV 空间、GPU memory、队列长度或 P/D 能力不足时，继续接收请求会造成排队爆炸和 OOM。Admission control 可以按预计 token 数、上下文长度、优先级和租户配额拒绝、延迟或降级请求，并保护已有 SLO。

85. TTFT、ITL、TPOT 和吞吐分别是什么?

答：TTFT 是从请求到首 token 的时间，主要受排队、Prefill 和首轮调度影响；ITL/TPOT 是后续 token 间隔，主要反映 Decode；吞吐通常以 output tokens/s 或 request/s 表示。评测时要明确是否包含网络、tokenizer、排队和 Prefill 时间。

86. 调度器如何兼顾吞吐、公平性和延迟?

答: 可以使用 token budget、最大请求数、优先级、年龄补偿、Decode 保证和 Prefill chunk 上限。只追求吞吐会让长请求或低优先级请求饿死，只追求低延迟会降低大 GEMM 效率，需要按业务 SLO 做策略搜索。

87. Prefix Cache 如何淘汰?

答: 常见策略包括 LRU、按 block 热度、按 prefix 命中收益和按租户配额淘汰。Hybrid 模型要把 KV、GDN state、conv state 作为一致性资源管理，不能只淘汰 KV 而保留一个不可恢复的 state 引用。

88. Prefix Cache 最容易出现哪些正确性问题?

答: 媒体内容不同但占位符相同、LoRA/模型版本变化、位置参数变化、状态边界不一致、取消请求后共享 state 被修改，都可能造成错误命中。缓存条目应包含内容 identity、版本、布局、边界和 ready event，并通过完整计算与命中恢复结果对比验证。

89. 多模态 encoder cache 的 key 如何设计?

答: 至少包含媒体内容 hash、媒体类型、分辨率/帧采样、processor 参数、vision encoder 版本、dtype 和适配器版本。Processor cache 与 encoder cache 可以分层管理，视觉 embedding 命中后仍要正确映射到文本 token、位置编码和语言模型 Prefix 边界。

90. AI Infra 需要监控哪些指标?

答: 请求侧监控 QPS、排队、TTFT、ITL、P99、错误和取消；模型侧监控 batch token 数、KV 使用率、cache hit、state 数量和 acceptance rate；硬件侧监控 GPU 利用率、HBM、带宽、温度、功耗、通信和最慢 rank。指标应带模型、版本、dtype、TP/EP 和租户标签。

91. 如何排查容器主机内存持续增长?

答: 先用 `docker stats` 区分容器 RSS、cache 和限制，再用 `docker top`、`ps`、`/proc/meminfo`、heap profiler 或 tracemalloc 定位进程、线程、队列、连接和临时文件。还要检查日志轮转、page cache、pinned memory 和重复模型加载，不能把 Docker 镜像层本身当作进程泄漏。

92. 如何排查 GPU 显存持续增长?

答: 使用 `nvidia-smi`、`watch -n 1 nvidia-smi` 和进程级显存查询，结合请求量固定的最小复现。重点检查 KV Cache、Prefix Cache、GDN state、CUDA caching allocator、workspace、Graph buffer、异常回滚和请求取消路径；区分 allocator 高水位和仍有活跃引用。

93. 常用 Linux/Docker 命令有哪些?

答: 磁盘整体用 `df -h`，目录用 `du -sh`，Docker 资源用 `docker system df` 和 `docker stats`；创建容器用 `docker run`，进入运行容器用 `docker exec -it`，查看日志用 `docker logs -f`。生产环境还要固定镜像 digest、GPU runtime、IPC、ulimit 和挂载权限。

94. 大模型权重加载慢如何定位?

答: 把时间拆成网络/磁盘读取、解压、反序列化、权重转换、分片分发、H2D 和通信初始化。使用本地 NVMe 缓存、safetensors 分片、并行读取和 rank-aware loading 可以改善启动，但要避免所有 rank 同时从远程存储拉完整模型。

95. `trust_remote_code` 有什么风险?

答: 它允许加载模型仓库中的自定义 Python 代码，可能执行任意导入、文件访问或网络操作。线上应使用可信仓库和固定 commit，审查模型代码，在隔离容器中运行，并尽量使用后端原生注册实现；若必须开启，应纳入安全评审和镜像扫描。

96. 如何保证 Hunyuan 推理结果可复现?

答: 固定模型 commit、tokenizer、chat template、后端版本、CUDA/驱动、dtype、量化 scale、采样参数和随机种子。还要记录是否启用 FlashAttention、CUDA Graph、MTP、Prefix Cache 和 CPU offload，因为不同 kernel 或 cache 路径可能造成小数差异和输出差异。

97. 如何做性能优化后的精度回归?

答: 先做逐层或逐算子输出对比，再做完整 logits、生成文本、长上下文、量化和并发回归。对 FP8/Int4、FlashAttention、融合 kernel、MTP 和 state 回滚，应分别设置容差和任务级指标，不能只测一个短 Prompt。

98. 如何做 AI Infra 的故障演练?

答: 模拟单卡退出、网卡抖动、All-to-All 超时、KV 空间不足、模型加载失败、请求取消和节点重启，验证超时、摘除、重试、状态丢弃/重算和告警链路。故障恢复必须保证不会复用半写入的 KV/state，也不能因为重试造成重复计费或请求乱序。

99. 如何做 Hunyuan 服务容量规划?

答: 按模型权重、KV/state、workspace、通信 buffer 和安全余量估算单副本显存，再结合输入/输出长度分布和目标并发估算 P/D/DP 副本数。容量模型至少包含平均和 P99 上下文、峰值 batch、cache 命中率、MoE 激活分布、GPU 利用率和网络带宽，而不是只按参数量除以显存。

100. 如果让你负责一次 Hunyuan AI Infra 优化项目，你会如何推进?

答: 先固定模型、硬件、后端和业务 SLO，建立精度、吞吐、TTFT、ITL、显存和通信基线；再用时间线与算子 profiler 判断瓶颈是在调度、GEMM、Attention、MoE、通信、缓存还是输入流水线。随后按收益/风险排序做量化、Grouped/Fused GEMM、Continuous Batching、Chunked Prefill、PD、缓存和通信重叠实验，每轮同时做精度与稳定性回归，最终交付可复现配置、代码变更、指标报告和回滚方案。

AI Infra 高频进阶 50 问（101–150）：混元部署与系统实战

本节承接前面的 100 道基础与核心问题，重点讨论面试中容易继续追问的实现边界、故障定位、长上下文、MoE 通信、多模态服务和混元工程落地细节。

A. 混元模型落地追问（101–115）

101. Hunyuan-A13B 的细粒度 MoE 对部署有什么影响?

答：细粒度 MoE 意味着专家数量、路由粒度和 token dispatch 复杂度更高，不能只按总参数量估算性能。部署时要同时观察每 token 激活专家数、专家 token 分布、Grouped GEMM 形状、All-to-All 字节数和热点专家尾延迟。[S27]

102. Hunyuan-Large 的 CLA 是否等于减少了所有 KV Cache?

答：不等于。CLA 通过跨层复用或压缩部分 KV 表示降低缓存压力，但具体可复用范围由模型结构和实现决定；仍需为未压缩层、边界 block、位置相关数据和运行时元数据分配空间。[S28]

103. Hunyuan-Large 使用 CLA 后，Prefix Cache 该如何设计?

答：缓存条目不能只保存传统每层 K/V，还要记录 CLA 的跨层依赖、压缩表示、层映射和有效 token 边界。命中时必须保证引用的共享表示不会被后续请求原地修改，最好使用引用计数或 copy-on-write。[S28]

104. HunyuanImage-3.0 为什么需要同时管理文本和图像 token?

答：它采用统一多模态框架，文本 token 和图像相关 latent/token 可能共同进入主干计算。调度器需要分别估算文本序列、图像 latent 数量、视觉 encoder 和扩散路径的预算，不能用纯文本 token budget 代表整个请求成本。[S30]

105. HunyuanImage-3.0 的统一自回归与扩散路径如何协同?

答：自回归路径负责文本、推理或条件序列建模，扩散路径负责图像 latent 的逐步去噪。工程上要区分两类 cache、随机数状态、采样 step 和显存峰值，并明确哪些 hidden/condition 可以跨 step 复用。[S30]

106. HunyuanVideo 的 Causal 3D VAE 为什么重要?

答：它在时间和空间上压缩视频，把原始像素转换成更少的 latent token，减少后续 DiT 的序列长度和计算量。VAE 也可能成为显存或解码瓶颈，因此应把 VAE 编码、DiT 采样和 VAE 解码分开 profiling。[S29]

107. HunyuanVideo 的 dual-stream 到 single-stream 结构有什么工程含义?

答：前段可以分别处理视频和文本 token，减少早期模态互相干扰；后段合并后进行跨模态融合。并行时要分别考虑两路 token 的 shape、位置编码、通信边界和合流点，不能简单把它看成一个普通文本 Transformer。[S29]

108. HunyuanVideo 1.5 的 SSTA 优化适合什么场景?

答: SSTA 通过选择性和滑动 tile 机制减少长视频中的冗余时空 token，长帧数、高分辨率场景更可能受益。短视频或小分辨率场景中，mask 生成和索引开销可能抵消部分收益，应按帧数、分辨率和 diffusion step 测试。[S31]

109. 混元官方性能数据为什么不能直接作为客户承诺?

答: 官方数据通常固定了 GPU、dtype、模型版本、输入长度、batch、并行配置和采样参数。客户负载可能包含长短请求混合、不同并发、Prefix Cache、图像/视频输入和不同 SLO，因此必须复现同工况并给出 P99，而不是只引用峰值 tokens/s。[S27–S31]

110. 混元模型适配自研硬件时，应该先适配模型还是先适配算子?

答: 先定义模型状态和算子语义，再做最小可运行路径，确认单卡 logits、权重映射和状态更新正确，然后替换高频算子。直接先追求融合 kernel 容易把布局错误、量化误差和调度问题混在一起，后续难以定位。

111. 如何检查 Hunyuan 模型的 config 是否与权重匹配?

答: 核对层数、hidden size、head 数、KV head 数、专家数、激活专家数、vocab size、RoPE 参数、量化 scale 和权重 key。加载后再逐层检查参数 shape、dtype、device 和 shard 范围，并用固定短输入对比参考实现 logits。

112. 模型代码为什么要固定 commit 而不是只固定仓库名?

答: 同一仓库的 `main` 分支可能改变模型类、权重映射、算子接口和默认配置。固定 commit、镜像 digest 和依赖 lockfile，才能复现性能与精度；线上升级应通过灰度和回滚验证，而不是直接拉最新代码。[S27、S28]

113. Hunyuan 的 tokenizer/chat template 为什么会影响性能?

答: tokenizer 决定输入 token 数，chat template 决定 system、角色、工具调用和多轮历史的额外 token。额外 token 会增加 Prefill、KV/state 和首 token 延迟；模板变化还会使 Prefix Cache key 失效，因此应把模板版本纳入基线和缓存身份。

114. 长上下文模型为什么仍可能在短输入下 OOM?

答: 服务通常按最大上下文、最大 batch、KV block、CUDA Graph buffer、MoE workspace 和通信 buffer 预留资源。即使当前请求很短，静态预分配或错误的 `max_model_len`、显存比例和并行配置也可能导致启动或运行时 OOM，需要区分预留、活跃分配和碎片。

115. 如何为混元建立“质量—性能—成本”三维基线?

答: 质量侧记录任务准确率、生成一致性、图像/视频质量和长上下文召回；性能侧记录 TTFT、ITL、吞吐、P99、显存和通信；成本侧记录 GPU 数、GPU 时间、能耗和实例利用率。只有三者绑定在同一模型版本、负载和精度下，才有配置决策价值。

B. 长上下文、缓存与投机执行（116–130）

116. 长上下文的主要成本是 KV Cache 还是 Attention 计算?

答： Decode 阶段常见主要成本是历史 KV 读取和权重读取，Prefill 阶段则可能由大矩阵和 Attention 计算主导。具体要看 batch、KV head 数、序列长度、FlashAttention、GQA/MLA 和硬件带宽，不能固定归因。

117. KV Cache block size 如何选择?

答： block 太小会增加页表、元数据和 kernel 索引开销，block 太大会增加尾部浪费并降低 Prefix Cache 命中粒度。应结合平均请求长度、长尾长度、GQA head 数、state 边界和后端 kernel 的访问模式做实测。

118. Prefix Cache 的 hash key 为什么不能只包含 token ID?

答： 相同 token 序列在不同模型、LoRA、视觉输入、位置参数、量化版本或 chat template 下可能产生不同 hidden/state。key 至少要包含内容、模型和适配器版本、媒体身份、位置相关配置、dtype/layout 以及状态表示版本。

119. Prefix Cache 命中时如何处理部分命中?

答： 找到最长共同前缀后，复用已提交的 KV/state，从命中边界继续计算剩余 token。若 Attention 命中长度和 GDN/conv state 边界不一致，应退回到共同安全边界，而不是只修改一个命中长度字段。[S12、S13]

120. Prefix Cache 的 state 如何防止被后续 Decode 修改?

答： 缓存中的 state 必须视为只读快照。命中后可以复制到请求独占槽，或使用 copy-on-write；异步传输和回收要用 event/refcount 保证写入完成、读取结束后再释放底层 buffer。

121. ReplaySSM 对 Prefix Cache 有什么影响?

答： 如果 state 由 checkpoint 加 replay 记录组成，缓存条目必须保存完整逻辑状态，或者在插入前执行 materialize。只缓存 checkpoint tensor 会丢失尚未 fold 的更新，导致命中后输出或后续 state 错误。[S4、S5]

122. 投机解码回滚时要回滚哪些内容?

答： 至少包括输出 token、KV Cache、GDN/SSM state、conv state、MTP/draft 状态、位置计数和请求的 block table。实现应先提交 accepted prefix，再截断 rejected suffix；任何一个状态未回滚都会造成下一轮结果漂移。

123. Draft Model 和 Target Model 共卡部署有什么风险?

答： 共卡可以减少通信，但 draft 会与 target 争抢显存、SM、带宽和调度时隙，接受率低时可能得不偿失。应比较 draft 单独耗时、verify 耗时、额外显存、每轮接受 token 和端到端 ITL，而不是只比较模型大小。

124. MTP verify 为什么适合融合算子?

答： verify 通常同时处理多个候选位置，包含状态读取、投影、比较和 accepted prefix 更新，拆成许多小算子会增加 launch 和中间内存。融合可以减少中间读写，但必须优先保证 accepted/rejected 分支、state layout 和精度语义正确。

125. 如何设置 speculative decoding 的 draft step?

答： 先在 2、3、4、5 等小范围扫描，记录 acceptance rate、每轮接受 token、draft/verify 时间、ITL 和吞吐。更大的 draft step 可能增加潜在收益，也会扩大错误累积和回滚成本，最优值依赖任务和并发。

126. 多模态请求的 encoder budget 为什么要独立?

答： 一张高分辨率图片或多帧视频的 encoder 成本不能用语言 token 数表示。调度器应分别限制文本 token、视觉 token、encoder batch 和 GPU memory，避免大量图片请求挤占 Decode 或让单轮 encoder 计算超出延迟预算。

127. Processor Cache 和 Encoder Cache 应该如何分层?

答： Processor Cache 放 CPU 可复用的解码/Resize/抽帧结果，容量较大但数据格式可能与 GPU 不同；Encoder Cache 放视觉 embedding，复用价值高但显存成本大。可以按热度和大小分层淘汰，并把预处理版本和视觉模型版本纳入 key。

128. 如何验证视觉 Encoder Cache 没有串图?

答： 使用两张内容不同但都包含相同 `<image>` 占位符的图片做交叉请求，比较视觉 embedding、语言 logits 和最终输出。还要测试分辨率、抽帧、processor 版本、LoRA 和 cache 淘汰/重建，确保 key 不会碰撞。

129. 长上下文 Prefix Cache 会不会把显存都占满?

答： 会，因此需要容量上限、租户配额、热度策略和回收机制。缓存收益应按“节省的 Prefill 时间/占用显存”评估；低命中率或只使用一次的长前缀可能不值得常驻 GPU，可以放 CPU/NVMe 或直接淘汰。

130. 缓存命中率高但性能没有提升，可能为什么?

答： 可能只命中了较短前缀，state 恢复仍需大量重算；也可能命中后发生大规模 state copy、跨卡传输、同步等待或 cache lookup 开销。应拆分命中 token 数、节省的 kernel 时间、恢复/copy 时间和端到端排队时间。

C. MoE、并行与通信进阶（131–142）

131. MoE 的 token dispatch 为什么容易成为瓶颈?

答： Router 输出的专家 ID 需要排序、permute、按 rank 分桶和 All-to-All，数据布局通常不规则。即使专家 GEMM 很快，dispatch/combine、索引构造和通信也可能占主要时间，必须单独测量。

132. 如何优化 MoE 的 token permutation?

答：减少重复排序和拷贝，使用预分配 workspace、连续索引、融合 pack/unpack，并让 dispatch 的布局直接匹配 Grouped GEMM 输入。优化后要验证 token 顺序、top-k 权重、padding 和 combine 的数值一致性。

133. Expert Replication 解决什么问题?

答：对长期热点专家复制多个副本，让不同请求或不同 rank 分摊热点 token。代价是额外权重显存和路由复杂度；如果热点随 batch 快速变化，静态复制可能收益不稳定，应结合在线统计和拓扑放置。

134. MoE 容量因子是什么?

答：容量因子限制每个专家在一个 batch 中最多接收的 token 数，避免单个专家无限膨胀。容量过小可能丢弃或重路由 token，影响质量；容量过大则增加显存和尾延迟，推理配置需要结合真实 token 分布选择。

135. TP、EP、DP 的组合如何确定?

答：先满足单副本权重和 KV/state 的显存约束，再用 TP 降低单请求计算、EP 分摊专家、DP 增加副本吞吐。固定总卡数后比较多组因式分解，同时测通信、P99、利用率和负载均衡，不能只按总参数量选并行维度。

136. TP 规模增大为什么可能变慢?

答：矩阵被切得更小后 GEMM 效率下降，All-Reduce/Reduce-Scatter 相对占比上升，跨节点链路也可能成为瓶颈。尤其 Decode 的 `M` 较小时，扩大 TP 更容易把计算变成通信受限。

137. EP 通信如何与专家 GEMM 做流水?

答：把 dispatch 后的 token 按 chunk 放入双缓冲，一块到达就启动对应 Grouped GEMM，同时下一块继续 All-to-All。需要明确 chunk 的 token 顺序、专家索引、stream/event 依赖和 combine 的回写位置，避免为了同步而失去重叠。

138. 如何判断通信是瓶颈而不是 GEMM?

答：时间线中通信区间占比高、GPU kernel 之间出现等待，且替换更快 GEMM 对端到端收益很小，通常说明通信受限。还要比较实际链路带宽、消息大小、collective 算法、最慢 rank 和拓扑，不能只看 NCCL kernel 名称。

139. All-to-All 为什么比 All-Reduce 更难优化?

答：All-to-All 的每个源 rank 向不同目标 rank 发送不同数据，消息大小和目的地可能随路由变化，容易产生不均匀和碎片化。MoE 还叠加 token 排序、专家热点和动态 shape，需要同时优化路由、拓扑、分桶和通信算法。

140. 如何调试多机通信初始化失败?

答：先用最小 collective 程序验证 GPU、网卡、端口、驱动和通信库，再逐步增加进程数和模型。检查 rank/world size、主节点地址、网卡选择、容器网络、NCCL/CNNL 日志、ulimit 和防火墙；不要直接在完整模型启动日志中猜原因。

141. 通信库环境变量为什么会影响性能?

答: 环境变量可能改变网卡选择、P2P、NCCL socket、IB、拓扑检测和算法阈值。调试时可以临时打开详细日志，但生产配置应固定并记录，避免不同节点继承了不一致的环境导致性能和稳定性漂移。

142. 如何避免并行配置导致死锁?

答: 所有 rank 必须以相同顺序进入 collective，分支条件不能让部分 rank 跳过通信；异步通信的 wait/event 也要覆盖所有数据依赖。故障、取消和异常路径必须释放或重建 process group，不能让一个 rank 提前退出而其他 rank 永久等待。

D. 服务和性能工程进阶（143–150）

143. 如何为多模型服务做 GPU 资源隔离?

答: 可以按模型副本、GPU 切分、进程组或队列隔离，并分别限制 batch、KV Cache 和并发。混元大模型通常不适合与小模型简单共享同一 GPU，需要评估权重常驻、cache 抖动、通信和 P99 互相影响。

144. 如何设计推理服务的 backpressure?

答: 当排队 token、KV 使用率、GPU 利用率或 P99 超过阈值时，系统应暂停接纳、降低最大新 token、延迟低优先级请求或返回可重试错误。backpressure 必须传播到网关和客户端，否则入口仍会无限堆积请求。

145. 为什么请求取消后仍可能占用显存?

答: 取消只是改变请求状态，异步 GPU kernel、通信、cache 引用和 state copy 可能尚未完成。需要等待相关 event、释放 block table/refcount、回收 GDN/conv state 和 draft buffer，并处理重复取消、超时和重试。

146. 如何定位 P99 突然升高?

答: 先按 TTFT、Decode ITL、排队、Prefill、通信、cache miss、GPU OOM 重试和最慢 rank 分桶。再按输入长度、输出长度、租户、模型版本、TP/EP、节点和时间段切片，避免把所有尾延迟都归因于一个 kernel。

147. 如何判断 CUDA Graph 值不值得在某个服务开启?

答: 统计 eager 下 host launch gap 占比、shape 分布和 graph 命中率，再计入 capture time、padding、graph memory 和 fallback。只有可复用 shape 足够多且提交开销占比明显时，Graph 才可能带来稳定收益。

148. 如何设计算子替换的灰度验证?

答: 先让新算子与旧算子在随机输入、边界 shape、不同 dtype 和异常值上做逐元素容差比较，再做单请求生成回归和小流量线上对比。灰度指标包括错误率、P99、显存、GPU 利用率和输出质量，发现异常应能按模型/shape/版本快速回滚。

149. 如何判断一次优化是真正的端到端收益？

答：必须同时看目标 kernel、所在阶段和完整请求链路：如果 kernel 快了 30%，但它只占总时间 5%，端到端最多只改善有限比例。优化报告应列出基线、改动、理论预期、profiling 证据、TTFT/ITL/吞吐/P99、显存和精度结果。

150. 面试中如何完整讲述一次混元 AI Infra 优化经历？

答：按“业务场景—模型与硬件—基线—瓶颈证据—方案—验证—交付”展开。例如先说明混元模型、输入输出长度和 TP/EP/PD 配置，再用时间线定位 MoE dispatch、GEMM、通信或 Cache 问题，实施量化、融合、重叠或调度改动，最后给出精度、吞吐、P99、显存和回滚方案。这样能体现你不仅会改算子，还能把模型、框架、硬件和服务指标串成闭环。

复习时最值得记住的 9 句话

1. nano-vLLM 与 vLLM 的差别主要是工程范围和策略复杂度，不是“有无优化”。
2. ReplaySSM 保存 checkpoint 与近期更新，减少完整递归状态的逐步写回。
3. Hybrid 缓存命中必须保证 KV、SSM、conv 对齐到同一前缀边界。
4. Hybrid PD 交接必须传完整逻辑状态；Replay buffer 未提交时也属于状态。
5. ViT 和 Decode 的瓶颈随 batch、长度和硬件变化，应看数据量和 profiler。
6. TP 切矩阵，DP 切请求副本，EP 切专家；实际进程组决定组合方式。
7. CUDA Graph 主要减少提交开销，融合还可能减少中间显存访问。
8. Continuous Batching 的核心是迭代级补位和 token 预算调度。
9. K3 适配已启动，当前约为同工况 H20 基线的 50%；性能结论必须绑定测试口径并通过 profiling 继续拆解。

资料与源码索引

以下均为项目官方仓库、官方文档、作者资料或原始论文。源码分支可能继续变化；issue/PR 仅用于说明设计和开发记录，不等同于已发布的稳定功能。

- [S1] nano-vLLM 项目说明：`https://github.com/GeeeekExplorer/nano-vllm`
- [S2] nano-vLLM Scheduler 源码：`https://github.com/GeeeekExplorer/nano-vllm/blob/main/nanovllm/engine/scheduler.py`
- [S3] vLLM V1 Scheduler 源码：`https://github.com/vllm-project/vllm/blob/main/vllm/v1/core/sched/scheduler.py`
- [S4] Dao AI Lab，ReplaySSM: Cache SSM Inputs, Not State，2026-06-15：`https://dao-lab.ai/blog/2026/replayssm/`
- [S5] SGLang ReplaySSM RFC 与实现记录：`https://github.com/sgl-project/sglang/issues/28511`；`https://github.com/sgl-project/sglang/pull/28695`；`https://github.com/sgl-project/sglang/pull/35544`。GDN backend：`https://github.com/sgl-project/sglang/blob/main/python/sglang/srt/layers/attention/linear/gdn_backend.py`
- [S6] Qwen3.5-397B-A17B 官方模型卡：`https://huggingface.co/Qwen/Qwen3.5-397B-A17B`
- [S7] vLLM Qwen3.5 模型实现：`https://github.com/vllm-project/vllm/blob/main/vllm/model_executor/models/qwen3_5.py`
- [S8] vLLM 多模态处理设计：`https://docs.vllm.ai/en/latest/design/mm_processing/`
- [S9] NVIDIA，Matrix Multiplication Background User's Guide：`https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html`
- [S10] NVIDIA Nsight Compute Profiling Guide / CLI：`https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html`；`https://docs.nvidia.com/nsight-compute/NsightComputeCli/index.html`
- [S11] NVIDIA Nsight Systems User Guide：`https://docs.nvidia.com/nsight-systems/UserGuide/index.html`
- [S12] vLLM Hybrid KV Cache Manager：`https://docs.vllm.ai/en/latest/design/hybrid_kv_cache_manager/`
- [S13] vLLM GDN 状态及算子实现：`https://github.com/vllm-project/vllm/blob/main/vllm/model_executor/layers/mamba/mamba_utils.py`；`https://github.com/vllm-project/vllm/blob/main/vllm/model_executor/layers/mamba/gdn/qwen_gdn_linear_attn.py`
- [S14] vLLM Disaggregated Prefilling：`https://docs.vllm.ai/en/latest/features/disagg_prefill/`
- [S15] vLLM Optimization and Tuning / Chunked Prefill：`https://docs.vllm.ai/en/latest/configuration/optimization/`

- **[S16] vLLM NIXL Connector 使用说明与兼容限制**：`https://docs.vllm.ai/en/latest/features/nixl_connector_usage/`；兼容矩阵：`https://docs.vllm.ai/en/latest/features/nixl_connector_compatibility/`；文档源码：`https://github.com/vllm-project/vllm/blob/main/docs/features/nixl_connector_usage.md`
- **[S17] FlashAttention 原始论文与实现**：`https://arxiv.org/abs/2205.14135`；`https://github.com/Dao-AILab/flash-attention`
- **[S18] Switch Transformers 原始论文**：`https://arxiv.org/abs/2101.03961`
- **[S19] vLLM Expert Parallel Deployment**：`https://docs.vllm.ai/en/latest/serving/expert_parallel_deployment/`
- **[S20] Megatron-LM 原始论文与官方仓库**：`https://arxiv.org/abs/1909.08053`；`https://github.com/NVIDIA/Megatron-LM`
- **[S21] NVIDIA CUTLASS Efficient GEMM / cuBLAS 文档**：`https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html`；`https://docs.nvidia.com/cuda/cublas/`
- **[S22] NVIDIA，Getting Started with CUDA Graphs**：`https://developer.nvidia.com/blog/cuda-graphs/`
- **[S23] vLLM CUDA Graphs 设计**：`https://docs.vllm.ai/en/latest/design/cuda_graphs/`
- **[S24] PyTorch torch.compiler 官方文档**：`https://docs.pytorch.org/docs/stable/torch.compiler.html`；`https://docs.pytorch.org/docs/stable/torch.compiler_dynamo_overview.html`
- **[S25] NVIDIA CUDA Best Practices Guide**：`https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html`
- **[S26] vLLM / PagedAttention 论文**：`https://arxiv.org/abs/2309.06180`
- **[S27] 腾讯混元 Hunyuan-A13B 官方仓库**：`https://github.com/Tencent-Hunyuan/Hunyuan-A13B`；包含 80B 总参数/13B 激活参数、256K 上下文、GQA、FP8/ GPTQ、TensorRT-LLM/vLLM/SGLang 部署说明。
- **[S28] 腾讯混元 Hunyuan-Large 官方仓库**：`https://github.com/Tencent/Tencent-Hunyuan-Large`；包含 389B 总参数/52B 激活参数、CLA、长上下文、H20 部署和 vLLM/TRT-LLM 资料。
- **[S29] 腾讯混元 HunyuanVideo 官方仓库**：`https://github.com/Tencent-Hunyuan/HunyuanVideo`；包含 Causal 3D VAE、Dual-stream/Single-stream、xDiT/USP、Ulysses/Ring、FP8 与显存说明。
- **[S30] 腾讯混元 HunyuanImage-3.0 官方仓库**：`https://github.com/Tencent-Hunyuan/HunyuanImage-3.0`；包含统一自回归多模态架构、64 Experts/8 active、FlashInfer、FlashAttention 和 vLLM 加速。
- **[S31] HunyuanVideo 1.5 技术报告**：`https://arxiv.org/html/2511.18870v1`；包含 8.3B DiT、SSTA、视频超分、feature caching、torch.compile 和 offload 分析。
- **[S32] 腾讯 AngelSlim 量化工具**：`https://github.com/Tencent/AngelSlim`；用于混元模型的 FP8、INT8、GPTQ/W4A16 等压缩与部署准备。

- **[S33] vLLM 官方文档**：`https://docs.vllm.ai/en/latest/`；调度、PagedAttention、量化、Prefix Cache、PD 分离和并行部署资料。
- **[S34] SGLang 官方文档**：`https://docs.sglang.io/`；运行时、Radix Cache、连续批处理、投机解码和多模型部署资料。
- **[S35] NVIDIA TensorRT-LLM 官方仓库**：`https://github.com/NVIDIA/TensorRT-LLM`；量化、Plugin、通信和高性能 LLM 推理资料。
- **[S36] PyTorch `torch.compile` 官方文档**：`https://docs.pytorch.org/docs/stable/torch.compiler.html`；Dynamo、Inductor、图捕获和编译边界。
- **[S37] NVIDIA CUDA Graphs 文档**：`https://developer.nvidia.com/blog/cuda-graphs/`；图捕获、replay 与提交开销说明。
- **[S38] NVIDIA Nsight Systems 文档**：`https://docs.nvidia.com/nsight-systems/UserGuide/index.html`；端到端时间线、CPU/GPU/通信分析。
- **[S39] NVIDIA Nsight Compute 文档**：`https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html`；kernel、Tensor Core、带宽、寄存器和 stall 分析。
- **[S40] FlashInfer 官方仓库**：`https://github.com/flashinfer-ai/flashinfer`；推理 Attention、KV Cache 和 MoE kernel 资料。
- **[S41] xDiT 官方仓库**：`https://github.com/xdit-project/xDiT`；DiT 的序列并行、Ulysses/Ring 和多 GPU 推理资料。
- **[S42] FlashAttention 官方仓库**：`https://github.com/Dao-AILab/flash-attention`；IO-aware Attention 实现与基准。
- **[S43] NVIDIA cuBLAS 官方文档**：`https://docs.nvidia.com/cuda/cublas/`；GEMM、cuBLASLt 和矩阵计算接口。